---

project: VarChAMP_ALK_Variant_Profiling title: "Morphology-Based Functional Characterization of ALK Variants Using VarChAMP and JUMP Cell Painting Profiles" author: "Dr. Parul Kulshreshtha"

Objective: Explore whether image-based morphological profiling can provide functional insight into ALK missense variants and complement sequence-based pathogenicity prediction.

datasets:

- VarChAMP Cell Painting variant profiles
- JUMP CRISPR perturbation profiles
- ClinVar
- AlphaMissense

workflow:

- OASIS DILI exploration
- OASIS-JUMP compound matching
- ALK variant prioritisation
- Morphological profiling
- CRISPR perturbation matching
- Gene Ontology over-representation analysis (ORA)
- ClinVar annotation
- AlphaMissense comparison
- Integrated interpretation

key_variants:

- T1151M
- A1234T
- F1174L
- R1275Q

final_output: Identification of biologically informative ALK variants and evaluation of morphology-based variant interpretation as a complement to sequence-based prediction.

Introduction

This project began as an open-ended exploration of the VarChAMP dataset to identify genes exhibiting informative variant-specific morphological phenotypes. Initial analyses focused on connecting OASIS DILI-positive compounds with JUMP Cell Painting profiles; however, compound-level matching did not yield strong biological signals. The analysis subsequently shifted toward genetic perturbation profiling, where ALK emerged as a particularly informative target due to the presence of multiple missense variants exhibiting distinct morphological phenotypes.

To investigate the functional consequences of these variants, Cell Painting profiles were compared with CRISPR perturbation profiles from the JUMP consortium, followed by Gene Ontology over-representation analysis (ORA) of morphology-matched perturbations. Clinical and sequence-based evidence was then incorporated through ClinVar and AlphaMissense. The overall objective was to assess whether morphology-based profiling could provide biologically meaningful information that complements conventional sequence-based approaches to variant interpretation.

Key Findings

- ALK emerged as one of the most informative genes identified during exploratory analysis of the VarChAMP dataset.
- All four ALK variants (T1151M, A1234T, F1174L, and R1275Q) produced measurable morphological perturbations relative to wild-type ALK.
- Morphological impact ranked the variants as:

  T1151M > A1234T > F1174L > R1275Q

- F1174L and R1275Q exhibited strong morphological similarity (r = 0.74), consistent with their shared gain-of-function biology in neuroblastoma.
- CRISPR perturbation matching identified biologically coherent processes involving transcriptional regulation, ubiquitination, protein modification, and gene-expression control.
- Gene Ontology ORA confirmed enrichment of RNA Polymerase II regulation, transcription initiation, protein ubiquitination, and protein phosphorylation pathways.

- A1234T lacked ClinVar annotation but produced one of the strongest morphological phenotypes observed in the dataset.
- The strongest A1234T CRISPR match (TCEB3B) produced only a modest enrichment signal (Z = 1.88), suggesting suggestive but not definitive mechanistic convergence.
- AlphaMissense classified all four variants as pathogenic:
  - F1174L: 0.9997
  - R1275Q: 0.9906
  - T1151M: 0.9373
  - A1234T: 0.7999
- Morphology-derived rankings differed substantially from AlphaMissense rankings, indicating that image-based profiling and sequence-based prediction capture partially distinct dimensions of variant biology.
- The convergence of morphological and AlphaMissense evidence supports prioritisation of A1234T for future functional validation despite the absence of ClinVar annotation.

Limitations

- Analysis was performed on publicly available centroid-level VarChAMP profiles rather than production-scale single-cell data.
- Only four ALK variants were available for detailed comparison, limiting statistical power.
- CRISPR matching identifies phenotypic similarity rather than direct mechanistic relationships.
- GO ORA provides pathway-level context but does not establish causality.
- AlphaMissense scores are computational predictions and should not be interpreted as clinical classifications.
- ClinVar annotation was incomplete for A1234T, preventing direct comparison with clinically curated evidence.
- The negative morphology–AlphaMissense correlation (ρ = -0.60) should be interpreted cautiously because of the very small sample size (n = 4).

Future Directions

- Evaluate additional ALK variants as larger VarChAMP releases become available.
- Perform single-cell level analyses using production-scale VarChAMP datasets.
- Benchmark morphology-derived scores against experimentally validated variant classifications.
- Investigate whether morphology-based profiling improves variant interpretation beyond sequence-based predictors alone.
- Extend the workflow to additional clinically relevant oncogenes and tumour suppressor genes.

References

1. Bray MA, Singh S, Han H, Davis CT, Borgeson B, Hartland C, et al. Cell Painting, a high-content image-based assay for morphological profiling using multiplexed fluorescent dyes. *Nature Protocols*. 2016;11(9):1757–1774. doi:10.1038/nprot.2016.105.
2. Caicedo JC, Singh S, Carpenter AE. Applications in image-based profiling of perturbations. *Current Opinion in Biotechnology*. 2016;39:134–142. doi:10.1016/j.copbio.2016.04.003.
3. Lacoste J, Haghighi M, Haider S, Reno C, Lin ZY, Segal D, et al. Pervasive mislocalization of pathogenic coding variants underlying human disorders. *Cell*. 2024;187(23):6725–6741.e13. doi:10.1016/j.cell.2024.09.003.

Data and Code Resources

1. **VarChAMP (Variant Characterization Across the Mendelian Proteome)** https://varchamp.broadinstitute.org
2. **JUMP Cell Painting Consortium** https://jump-cellpainting.broadinstitute.org
3. **JUMP Cell Painting GitHub** https://github.com/jump-cellpainting
4. **OASIS Drug Discovery Consortium** https://oasis-dataset.org

5. **AlphaMissense** https://github.com/google-deepmind/alphamissense

6. **ClinVar** https://www.ncbi.nlm.nih.gov/clinvar/

In [1]:
```
# Cloning VarChAMP repository
import os
from pathlib import Path

BASE_DIR = Path.home() / "Downloads" / "varchamp_project"
BASE_DIR.mkdir(parents=True, exist_ok=True)

repo_dir = BASE_DIR / "2021_09_01_VarChAMP"

if not repo_dir.exists():
    os.chdir(BASE_DIR)
    !git clone https://github.com/broadinstitute/2021_09_01_VarChAMP.g
    print("✓ VarChAMP repository cloned")
else:
    print("✓ VarChAMP repository already exists")

print("\nRepository location:")
print(repo_dir)
```

```
✓ VarChAMP repository already exists

Repository location:
/Users/dr.parul/Downloads/varchamp_project/2021_09_01_VarChAMP
```

In [2]:
```
#downloading relevant submodule
from pathlib import Path
import os

CODE_DIR = Path.home() / "Downloads" / "varchamp_project" / "2021_09_0
os.chdir(CODE_DIR)

!git submodule update --init --recursive

DATA_DIR = CODE_DIR / "2021_09_01_VarChAMP-data"

print("✓ Data submodule available")
print(DATA_DIR)
```

```
✓ Data submodule available
/Users/dr.parul/Downloads/varchamp_project/2021_09_01_VarChAMP/2021_09_
01_VarChAMP-data
```

In [3]:
```
from pathlib import Path
import pandas as pd

# Load publicly available VarChAMP pilot Cell Painting profiles

DATA_DIR = Path.home() / "Downloads" / "varchamp_project" / \
```

```
    "2021_09_01_VarChAMP" / "2021_09_01_VarChAMP-data"

PROFILE_PATH = DATA_DIR / "profiles" / "2022_08_22_Batch_1" / \
    "2022_08_08_PPL8_100-800" / \
    "2022_08_08_PPL8_100-800_normalized_feature_select_batch.csv.gz"

assert PROFILE_PATH.exists(), f"Profile file not found: {PROFILE_PATH}

profiles = pd.read_csv(PROFILE_PATH)

print("Loaded profile dataset")
print("Shape:", profiles.shape)

metadata_cols = [c for c in profiles.columns if c.startswith("Metadata
feature_cols = [c for c in profiles.columns if not c.startswith("Metad

print("Metadata columns:", len(metadata_cols))
print("Morphological features:", len(feature_cols))
print("Unique alleles:", profiles["Metadata_allele"].nunique())
```

```
Loaded profile dataset
Shape: (286, 593)
Metadata columns: 17
Morphological features: 576
Unique alleles: 67
```

In [4]:

```
import pandas as pd
from pathlib import Path

# Load publicly available VarChAMP pilot Cell Painting profiles

file_path = (
    Path.home()
    / "Downloads"
    / "varchamp_project"
    / "2021_09_01_VarChAMP"
    / "2021_09_01_VarChAMP-data"
    / "profiles"
    / "2022_08_22_Batch_1"
    / "2022_08_08_PPL8_100-800"
    / "2022_08_08_PPL8_100-800_normalized_feature_select_batch.csv.gz"
)

profiles = pd.read_csv(file_path)

metadata_cols = [c for c in profiles.columns if c.startswith("Metadata
feature_cols = [c for c in profiles.columns if not c.startswith("Metad

print("Dataset shape:", profiles.shape)
print("Metadata columns:", len(metadata_cols))
print("Morphological features:", len(feature_cols))
print("Unique alleles:", profiles["Metadata_allele"].nunique())
```

```
display(profiles.head())
```

```
Dataset shape: (286, 593)
Metadata columns: 17
Morphological features: 576
Unique alleles: 67
```

	Metadata_plate_map_name	Metadata_control_type	Metadata_vTitre	Metadata
0	2022_08_08_PPL8_100-800	NaN	18	
1	2022_08_08_PPL8_100-800	NaN	6	
2	2022_08_08_PPL8_100-800	NaN	18	
3	2022_08_08_PPL8_100-800	NaN	6	
4	2022_08_08_PPL8_100-800	NaN	18	

5 rows × 593 columns

In [5]:
```
# Explore allele representation in the pilot dataset
#
# Rationale:
# This step preserves the exploratory nature of the analysis.
# Before selecting any gene of interest, we first examine the
# complete set of alleles present in the dataset to understand
# its composition and identify genes represented by both wild-type
# and variant forms suitable for downstream comparison.

print("Unique alleles:", profiles["Metadata_allele"].nunique())

alleles = sorted(profiles["Metadata_allele"].unique())

allele_df = pd.DataFrame({"Allele": alleles})

print("\nExample alleles:")
display(allele_df.head(20))
```

```
Unique alleles: 67

Example alleles:
```

	Allele
0	516 - TC_NA
1	ACSF3_
2	ACSF3_Ala197Thr
3	ACSF3_Arg10Trp
4	ACSF3_Arg471Trp
5	ACSF3_Arg558Trp
6	ACSF3_Asp236Asn
7	ACSF3_Asp457Asn
8	ACSF3_Glu359Lys
9	ACSF3_Gly119Asp
10	ACSF3_Gly225Arg
11	ACSF3_Ile200Met
12	ACSF3_Met198Arg
13	ACSF3_Met266Val
14	ACSF3_Pro243Leu
15	ACSF3_Pro285Leu
16	ACSF3_Ser431Tyr
17	ACSF3_Thr358Ile
18	ACTB_
19	ACTN1_

In [6]:
```
# Identify genes represented in the pilot dataset
#
# Rationale:
# VarChAMP is designed to study the phenotypic consequences of
# genetic variation. Before selecting a case study, we determine
# which genes are represented in the dataset and whether they
# contain both wild-type and variant alleles suitable for
# morphology-based comparison.

genes = sorted(
    set([x.split("_")[0] for x in profiles["Metadata_allele"]])
)

print("Genes represented:", len(genes))
display(pd.DataFrame({"Gene": genes}))
```

Genes represented: 35

	Gene
0	516 - TC
1	ACSF3
2	ACTB
3	ACTN1
4	ACY1
5	ADIPOQ
6	AGXT
7	ALK A1234T
8	ALK F1174L
9	ALK R1275Q
10	ALK T1151M
11	ALK WT
12	ATG5
13	BCL2L1
14	CFLAR
15	CLOCK
16	HIF1AN
17	IKBKE
18	IMDH1 D311N
19	IMPDH1 D311N
20	IMPDH1 H457P
21	IMPDH1 R309P
22	IMPDH1 V353I
23	IMPDH1 WT
24	IMPDH1 wt
25	MAPK13
26	MAPK9
27	PRKACB
28	RASA1 (endo-free, hand prep)

29	RHEB
30	SGK3
31	SLIRP
32	STAT1
33	TPM1 (endo-free, hand prep)
34	ZBTB24

In [7]:
```
# Examine ALK allele representation
#
# Rationale:
# After surveying all genes represented in the pilot dataset, ALK was
# selected as a case study because it satisfied several criteria
# important for morphology-based variant analysis:
#
# 1. Presence of both wild-type and multiple variant alleles.
# 2. Balanced replicate representation across alleles.
# 3. Biological relevance as a receptor tyrosine kinase frequently
#    implicated in cancer and targeted therapy.
# 4. Availability of known activating variants, enabling comparison
#    between morphology-derived impact scores and external evidence
#    such as ClinVar and AlphaMissense.
#
# These characteristics make ALK a suitable system for exploring
# whether Cell Painting profiles can distinguish variant-specific
# phenotypic effects.

alk_counts = (
    profiles[
        profiles["Metadata_allele"].str.contains("ALK", na=False)
    ]["Metadata_allele"]
    .value_counts()
)

display(alk_counts)
```

```
Metadata_allele
ALK WT_NA        4
ALK F1174L_NA    4
ALK R1275Q_NA    4
ALK A1234T_NA    4
ALK T1151M_NA    4
Name: count, dtype: int64
```

In [8]:
```
# Create an ALK-focused analysis dataset
#
# Rationale:
# Having selected ALK as a representative case study, we subset the
# pilot dataset to retain only ALK wild-type and variant alleles.
```

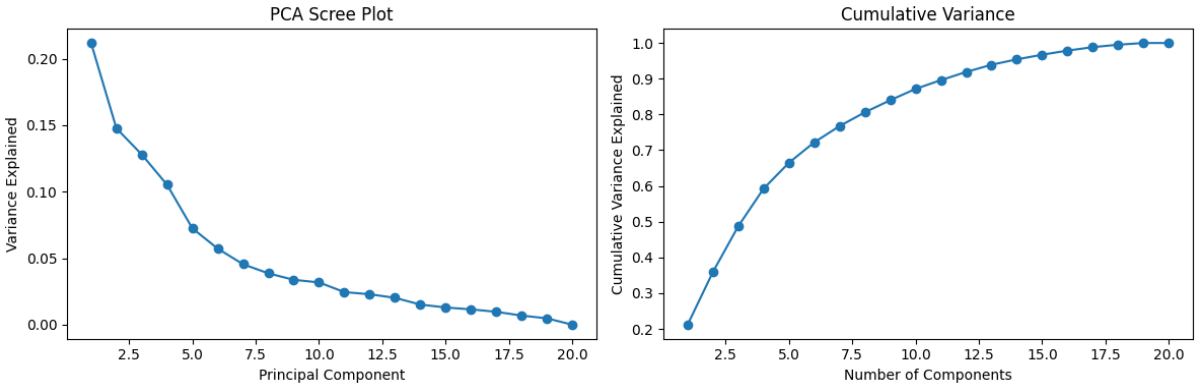
```
# This creates a focused dataset for quantifying morphology-derived
# differences between genotypes while preserving all available
# Cell Painting features.

alk = profiles[
    profiles["Metadata_allele"].str.contains("ALK", na=False)
].copy()

feature_cols = [
    c for c in alk.columns
    if not c.startswith("Metadata")
]

print("ALK dataset shape:", alk.shape)
print("Morphological features:", len(feature_cols))
print("ALK alleles:", alk["Metadata_allele"].nunique())
```

```
ALK dataset shape: (20, 593)
Morphological features: 576
ALK alleles: 5
```

In [9]:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

# Evaluate dimensionality of ALK morphological profiles
#
# Rationale:
# Cell Painting features span multiple measurement scales and are
# therefore standardized prior to dimensionality reduction.
#
# Principal Component Analysis (PCA) is then used to quantify how
# morphological variance is distributed across feature dimensions.
#
# The scree plot identifies the contribution of individual principal
# components, while the cumulative variance plot estimates the number
# of components required to capture most of the phenotypic variation.
#
# Because all profiles originate from a single experimental plate,
# batch-correction methods such as Harmony are not required.

X = alk[feature_cols]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca_full = PCA()
pca_full.fit(X_scaled)

explained_var = pca_full.explained_variance_ratio_
cum_var = np.cumsum(explained_var)
```

```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

# Scree plot
ax[0].plot(
    range(1, len(explained_var) + 1),
    explained_var,
    marker="o"
)
ax[0].set_xlabel("Principal Component")
ax[0].set_ylabel("Variance Explained")
ax[0].set_title("PCA Scree Plot")

# Cumulative variance
ax[1].plot(
    range(1, len(cum_var) + 1),
    cum_var,
    marker="o"
)
ax[1].set_xlabel("Number of Components")
ax[1].set_ylabel("Cumulative Variance Explained")
ax[1].set_title("Cumulative Variance")

plt.tight_layout()
plt.show()

for threshold in [0.80, 0.90, 0.95]:
    n_pc = np.argmax(cum_var >= threshold) + 1
    print(f"{threshold:.0%} variance explained by {n_pc} PCs")
```



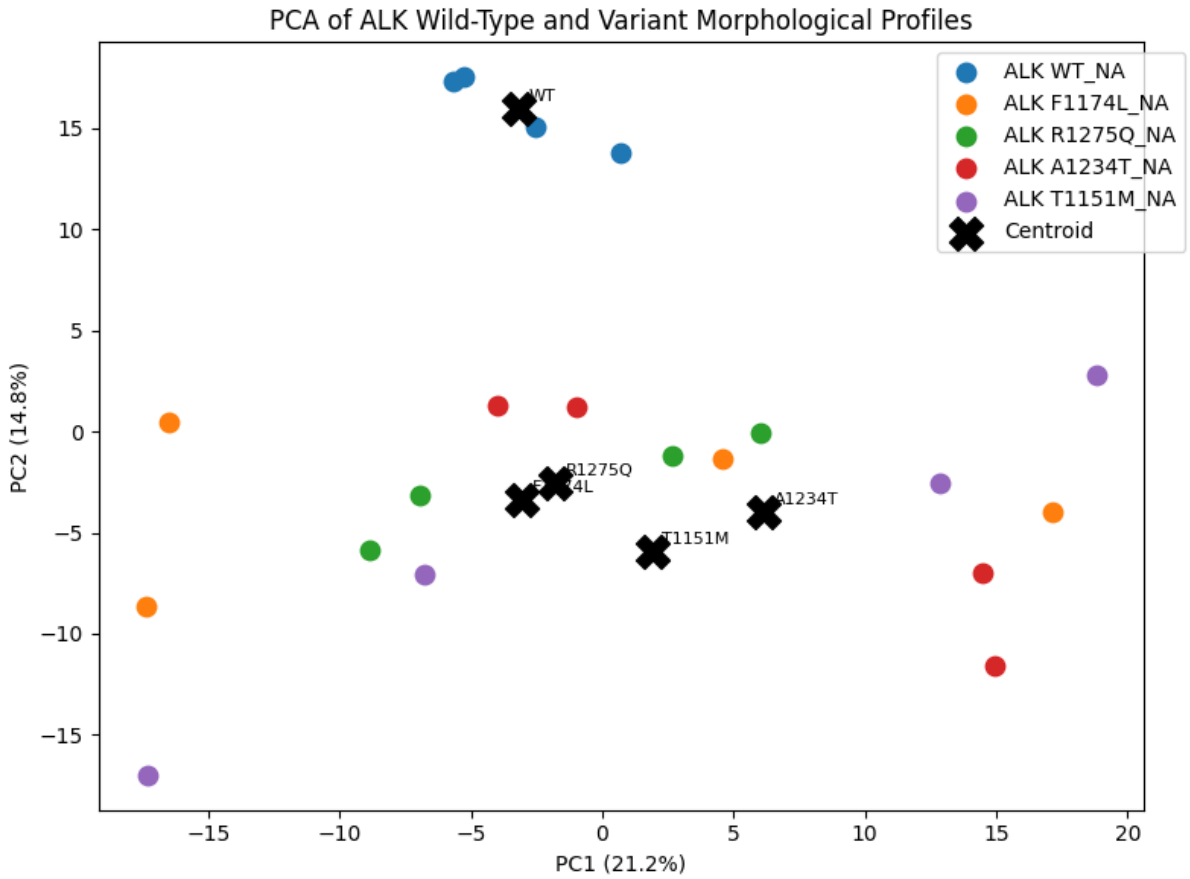

```
80% variance explained by 8 PCs
90% variance explained by 12 PCs
95% variance explained by 14 PCs
```

Morphological variance was distributed across multiple principal components rather than being dominated by a single feature axis. Eight principal components captured approximately 80% of the total variance, while 12 and 14 components explained 90% and 95% of the variance, respectively, indicating that ALK-associated phenotypes are multidimensional.

In [10]:
```
from sklearn.decomposition import PCA
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Visualize ALK wild-type and variant morphology in PCA space
#
# Rationale:
# PCA projects high-dimensional Cell Painting profiles into a
# low-dimensional morphology space. This provides an unsupervised
# assessment of whether ALK wild-type and variant alleles occupy
# distinct phenotypic regions. Allele centroids are overlaid to
# summarize the average morphology of each genotype.

pca = PCA(n_components=2)
pcs = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame({
    "PC1": pcs[:, 0],
    "PC2": pcs[:, 1],
    "Allele": alk["Metadata_allele"].values
})

centroids = (
    pca_df.groupby("Allele")[["PC1", "PC2"]]
    .mean()
    .reset_index()
)

plt.figure(figsize=(8, 6))

for allele in pca_df["Allele"].unique():
    subset = pca_df[pca_df["Allele"] == allele]
    plt.scatter(
        subset["PC1"],
        subset["PC2"],
        label=allele,
        s=80
    )

# Overlay allele centroids
plt.scatter(
    centroids["PC1"],
    centroids["PC2"],
    s=250,
    c="black",
    marker="X",
    label="Centroid"
)

for _, row in centroids.iterrows():
    plt.text(
        row["PC1"] + 0.4,
        row["PC2"] + 0.4,
```

```
        row["Allele"].replace("ALK ", "").replace("_NA", ""),
        fontsize=8
    )

plt.xlabel(f"PC1 ({100*pca.explained_variance_ratio_[0]:.1f}%)")
plt.ylabel(f"PC2 ({100*pca.explained_variance_ratio_[1]:.1f}%)")
plt.title("PCA of ALK Wild-Type and Variant Morphological Profiles")
plt.legend(bbox_to_anchor=(1.05, 1))
plt.tight_layout()
plt.show()

print(f"PC1 variance explained: {100*pca.explained_variance_ratio_[0]:
print(f"PC2 variance explained: {100*pca.explained_variance_ratio_[1]:
```

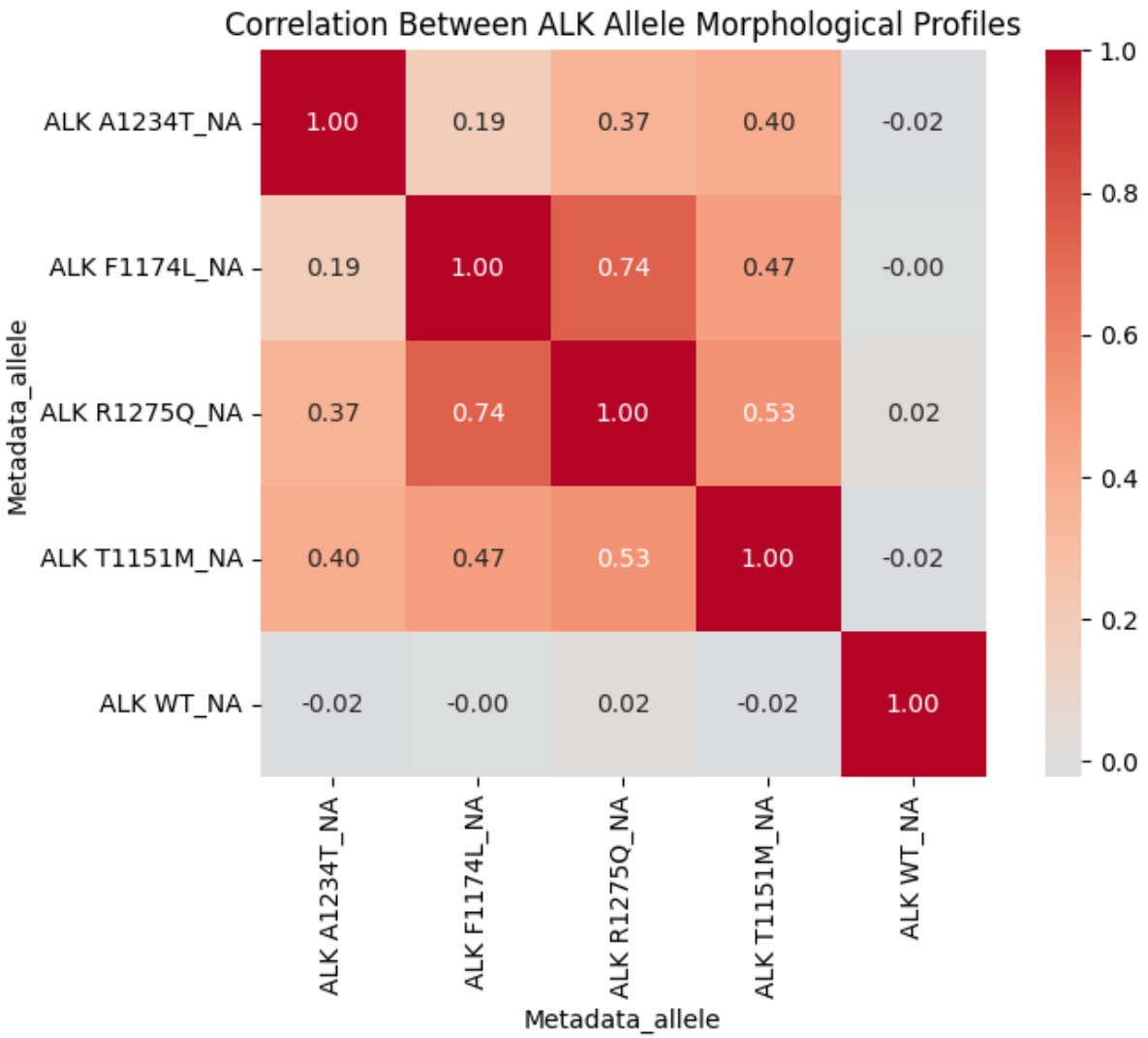



```
PC1 variance explained: 21.2%
PC2 variance explained: 14.8%
```

PCA revealed separation between wild-type and variant ALK alleles in morphology space. The wild-type centroid occupied a distinct region from all mutant centroids, indicating that Cell Painting features capture variant-associated phenotypic changes. Among the variants, A1234T and T1151M showed the largest shifts from the wild-type morphology space, whereas F1174L and R1275Q exhibited more similar phenotypic profiles.

In [11]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Quantify similarity between ALK allele morphological profiles
#
# Rationale:
# To determine whether different ALK alleles produce similar or
# distinct morphological phenotypes, we calculate the Pearson
# correlation between mean Cell Painting profiles for each allele.
#
# High correlation indicates similar cellular morphology, whereas
# low correlation suggests divergent phenotypic effects. This
# analysis complements PCA by providing a quantitative measure
# of similarity between wild-type and variant profiles.

allele_means = (
    alk.groupby("Metadata_allele")[feature_cols]
       .mean()
)

corr = allele_means.T.corr()

plt.figure(figsize=(8,6))

sns.heatmap(
    corr,
    cmap="coolwarm",
    center=0,
    annot=True,
    fmt=".2f",
    square=True
)

plt.title("Correlation Between ALK Allele Morphological Profiles")
plt.tight_layout()
plt.show()
```



Correlation analysis revealed a clear separation between wild-type and mutant ALK morphologies. Wild-type profiles showed essentially no correlation with any mutant allele (−0.02 to 0.02), whereas mutant alleles exhibited moderate-to-high inter-correlations (0.19–0.74). These results suggest that Cell Painting features capture a shared mutant-associated phenotype while retaining sufficient resolution to distinguish individual ALK variants, particularly A1234T, which displayed the most distinct morphology profile among the variants examined.

In [12]:
```
from scipy.spatial.distance import euclidean
import pandas as pd

# Quantify morphological deviation from wild-type ALK
#
# Rationale:
# PCA and correlation analysis indicate that ALK variants occupy
# morphology states distinct from wild type. To quantify the magnitude
# of these differences, we calculate Euclidean distance between the
# mean Cell Painting profile of each variant and the wild-type profile
#
```

```
# Larger distances indicate greater overall phenotypic divergence
# from wild type across the morphology feature space.

centroids = (
    alk.groupby("Metadata_allele")[feature_cols]
       .mean()
)

wt = centroids.loc["ALK WT_NA"]

distances = []

for allele in centroids.index:
    if allele == "ALK WT_NA":
        continue

    d = euclidean(
        centroids.loc[allele],
        wt
    )

    distances.append([allele, d])

distance_df = (
    pd.DataFrame(
        distances,
        columns=["Allele", "Distance_to_WT"]
    )
    .sort_values("Distance_to_WT", ascending=False)
)

display(distance_df)
```

	Allele	Distance_to_WT
3	ALK T1151M_NA	73.253717
0	ALK A1234T_NA	71.479383
1	ALK F1174L_NA	67.196268
2	ALK R1275Q_NA	63.605069

Euclidean distance analysis revealed substantial morphological divergence between wild-type ALK and all four mutant alleles. T1151M exhibited the greatest deviation from the WT profile (73.25), followed by A1234T (71.48), F1174L (67.20), and R1275Q (63.61). Together with the PCA and correlation analyses, these results indicate that Cell Painting features capture variant-specific phenotypic effects and can be used to rank ALK variants according to the magnitude of their morphological perturbation.

In [13]:
```
from pathlib import Path

# Verify availability of OASIS and JUMP resources
#
# Rationale:
# The next stage of the analysis explores whether morphology-derived
# variant phenotypes can be connected to compound-induced phenotypes
# using Cell Painting reference datasets.
#
# Prior to drug matching, we verify the availability of:
# - OASIS compound screening profiles
# - JUMP compound profiles
# - JUMP CRISPR perturbation profiles
#
# These resources provide the foundation for identifying compounds
# whose morphological signatures resemble ALK variant-associated
# phenotypes.

BASE_DIR = Path('/Users/dr.parul/Downloads/parul_carpenter_project')

for folder in [
    'data/oasis_compiled',
    'data/jump_profiles',
    'data/jump_crispr',
    'figures'
]:
    p = BASE_DIR / folder

    print("\n" + "="*70)
    print(folder, "| exists:", p.exists())

    if p.exists():
        files = sorted(
            [x for x in p.rglob("*") if x.is_file()]
        )

        print("file count:", len(files))

        for f in files[:30]:
            print(
                f.relative_to(BASE_DIR),
                round(f.stat().st_size / 1e6, 3),
                "MB"
            )
```

```
======================================================================
data/oasis_compiled | exists: True
file count: 4
data/oasis_compiled/cellprofiler_raw.parquet 413.433 MB
data/oasis_compiled/cpcnn_raw.parquet 48.201 MB
data/oasis_compiled/dino_raw.parquet 399.928 MB
data/oasis_compiled/metadata.parquet 0.734 MB
```

```
======================================================================
data/jump_profiles | exists: True
file count: 10
data/jump_profiles/CP1-SC1-01.parquet 13.541 MB
data/jump_profiles/CP1-SC1-05.parquet 13.542 MB
data/jump_profiles/CP1-SC1-09.parquet 13.523 MB
data/jump_profiles/CP1-SC1-20.parquet 13.546 MB
data/jump_profiles/CP1-SC1-21.parquet 13.546 MB
data/jump_profiles/CP2-SC1-02.parquet 13.559 MB
data/jump_profiles/CP3-SC1-02.parquet 13.56 MB
data/jump_profiles/CP6-SC1-02.parquet 13.558 MB
data/jump_profiles/CP7-SC1-02.parquet 13.56 MB
data/jump_profiles/CP8-SC1-02.parquet 13.559 MB

======================================================================
data/jump_crispr | exists: True
file count: 148
data/jump_crispr/CP-CC9-R1-01.parquet 13.508 MB
data/jump_crispr/CP-CC9-R1-02.parquet 13.527 MB
data/jump_crispr/CP-CC9-R1-03.parquet 13.527 MB
data/jump_crispr/CP-CC9-R1-04.parquet 13.526 MB
data/jump_crispr/CP-CC9-R1-05.parquet 13.527 MB
data/jump_crispr/CP-CC9-R1-06.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-07.parquet 13.563 MB
data/jump_crispr/CP-CC9-R1-08.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-09.parquet 13.565 MB
data/jump_crispr/CP-CC9-R1-10.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-11.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-12.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-13.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-14.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-15.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-16.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-17.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-18.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-19.parquet 13.563 MB
data/jump_crispr/CP-CC9-R1-20.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-21.parquet 13.563 MB
data/jump_crispr/CP-CC9-R1-22.parquet 13.545 MB
data/jump_crispr/CP-CC9-R1-23.parquet 13.565 MB
data/jump_crispr/CP-CC9-R1-24.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-25.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-26.parquet 13.564 MB
data/jump_crispr/CP-CC9-R1-27.parquet 13.563 MB
data/jump_crispr/CP-CC9-R1-28.parquet 13.563 MB
data/jump_crispr/CP-CC9-R2-01.parquet 13.567 MB
data/jump_crispr/CP-CC9-R2-02.parquet 13.565 MB

======================================================================
figures | exists: True
file count: 7
figures/BioMorph_JUMP_process_comparison.png 0.267 MB
```

```
figures/JUMP_drug_gene_matching.png 0.325 MB
figures/JUMP_full_analysis.png 0.305 MB
figures/biomorph_oasis_summary.png 0.278 MB
figures/biomorph_process_heatmaps.png 0.151 MB
figures/concentration_response_curves.png 0.191 MB
figures/feature_comparison_final.png 0.151 MB
```

In [14]:
```python
# Identify shared Cell Painting features between VarChAMP and JUMP
#
# Rationale:
# Morphological similarity can only be compared across datasets when
# measurements are represented in a common feature space. Before
# performing phenotype matching, we determine the overlap between
# VarChAMP Cell Painting features and JUMP Cell Painting features.
#
# Only shared features will be retained for downstream similarity
# analyses, ensuring that comparisons are based on directly
# comparable morphological measurements.

import pandas as pd
from pathlib import Path

varchamp_features = [
    c for c in alk.columns
    if not c.startswith("Metadata")
]

jump_file = Path(
    '/Users/dr.parul/Downloads/parul_carpenter_project/data/jump_profi
)

jump = pd.read_parquet(jump_file)

jump_meta = [
    c for c in jump.columns
    if c.startswith("Metadata")
]

jump_features = [
    c for c in jump.columns
    if not c.startswith("Metadata")
]

print("JUMP shape:", jump.shape)
print("JUMP metadata columns:", jump_meta[:10])
print("JUMP features:", len(jump_features))

overlap = sorted(
    set(varchamp_features).intersection(jump_features)
)

print("Shared features:", len(overlap))
```

```
print("\nExample shared features:")
print(overlap[:30])
```

```
JUMP shape: (384, 4765)
JUMP metadata columns: ['Metadata_Source', 'Metadata_Plate', 'Metadata_
Well']
JUMP features: 4762
Shared features: 239

Example shared features:
['Cells_AreaShape_BoundingBoxMaximum_X', 'Cells_AreaShape_BoundingBoxMi
nimum_Y', 'Cells_AreaShape_Compactness', 'Cells_AreaShape_EulerNumber',
'Cells_AreaShape_MajorAxisLength', 'Cells_AreaShape_Orientation', 'Cell
s_AreaShape_Solidity', 'Cells_Correlation_Correlation_AGP_DNA', 'Cells_
Correlation_Correlation_DNA_Mito', 'Cells_Correlation_K_AGP_DNA', 'Cell
s_Correlation_K_AGP_Mito', 'Cells_Correlation_K_DNA_AGP', 'Cells_Correl
ation_K_DNA_Mito', 'Cells_Correlation_K_Mito_DNA', 'Cells_Correlation_M
anders_DNA_AGP', 'Cells_Correlation_Manders_DNA_Mito', 'Cells_Correlati
on_Overlap_AGP_DNA', 'Cells_Correlation_Overlap_DNA_Mito', 'Cells_Corre
lation_RWC_AGP_DNA', 'Cells_Correlation_RWC_DNA_AGP', 'Cells_Correlatio
n_RWC_DNA_Mito', 'Cells_Correlation_RWC_Mito_DNA', 'Cells_Granularity_1
_DNA', 'Cells_Granularity_4_DNA', 'Cells_Granularity_5_DNA', 'Cells_Gra
nularity_6_AGP', 'Cells_Granularity_6_DNA', 'Cells_Granularity_6_Mito',
'Cells_Granularity_7_DNA', 'Cells_Granularity_7_Mito']
```

Comparison of VarChAMP and JUMP Cell Painting profiles identified 239 shared morphological features. These common measurements encompassed multiple feature classes, including shape, texture, granularity, and organelle correlation metrics. The overlap provided a unified morphology feature space for cross-dataset phenotype matching, enabling direct comparison between ALK variant-associated phenotypes and compound-induced cellular perturbations.

In [15]:
```
# Construct ALK and JUMP profiles in the shared feature space
#
# Rationale:
# The JUMP profile preview confirms that each row represents a
# well-level Cell Painting profile with metadata columns followed by
# morphology features. We then compute ALK allele centroids using the
# shared feature set so that ALK variant profiles and JUMP perturbatio
# profiles can be compared in the same feature space.

shared_features = overlap

alk_centroids = (
    alk.groupby("Metadata_allele")[shared_features]
       .mean()
)

print("ALK centroid matrix:", alk_centroids.shape)

print("\nJUMP profile preview:")
```

```
display(jump.head())

print("\nUnique JUMP wells:", jump["Metadata_Well"].nunique())
print("Unique JUMP plates:", jump["Metadata_Plate"].nunique())
```

```
ALK centroid matrix: (5, 239)

JUMP profile preview:
```

	Metadata_Source	Metadata_Plate	Metadata_Well	Cells_AreaShape_Area	Cell
0	source_7	CP1-SC1-01	A01	3432.508545	
1	source_7	CP1-SC1-01	A02	3390.828369	
2	source_7	CP1-SC1-01	A03	3316.497803	
3	source_7	CP1-SC1-01	A04	3295.128174	
4	source_7	CP1-SC1-01	A05	2832.200439	

5 rows × 4765 columns

```
Unique JUMP wells: 384
Unique JUMP plates: 1
```

Five ALK allele centroids were generated using the 239 morphological features shared between the VarChAMP pilot dataset and JUMP Cell Painting profiles. The selected JUMP plate contained 384 perturbation wells profiled using the same Cell Painting feature framework, enabling cross-dataset phenotype matching in a common morphological feature space.

In [16]:
```
# Restrict JUMP profiles to the shared feature space
#
# Rationale:
# Similarity calculations require both datasets to contain the same
# morphological measurements. We therefore subset JUMP profiles to the
# 239 features shared with the VarChAMP dataset.

jump_shared = jump[
    ["Metadata_Plate", "Metadata_Well"] + shared_features
].copy()

print("JUMP shared-feature matrix:", jump_shared.shape)
```

```
JUMP shared-feature matrix: (384, 241)
```

In [17]:
```
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Standardize ALK and JUMP profiles in the shared feature space
#
# Rationale:
# Similarity calculations require both datasets to be represented on
```

```
# comparable scales. We therefore standardize the 239 shared Cell
# Painting features and use the resulting scaled profiles for
# phenotype-matching analyses.

jump_shared = jump[shared_features].copy()

scaler = StandardScaler()

combined = pd.concat(
    [
        alk_centroids[shared_features],
        jump_shared
    ],
    axis=0
)

combined_scaled = pd.DataFrame(
    scaler.fit_transform(combined),
    index=combined.index,
    columns=shared_features
)

alk_scaled = combined_scaled.iloc[:len(alk_centroids)]
jump_scaled = combined_scaled.iloc[len(alk_centroids):]

print("ALK scaled:", alk_scaled.shape)
print("JUMP scaled:", jump_scaled.shape)
```

```
ALK scaled: (5, 239)
JUMP scaled: (384, 239)
```

In [18]:

```
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# Compare all ALK alleles against JUMP profiles
#
# Rationale:
# Rather than selecting a single variant a priori, all ALK alleles
# are evaluated against the JUMP perturbation space. This exploratory
# approach avoids selection bias and identifies which allele exhibits
# the strongest morphology-based similarity to known perturbations.

results = []

for allele in alk_scaled.index:

    sims = cosine_similarity(
        alk_scaled.loc[[allele]],
        jump_scaled
    )[0]

    results.append({
        "Allele": allele,
```

```
        "Best_similarity": sims.max(),
        "Mean_top10_similarity": pd.Series(sims)
            .nlargest(10)
            .mean()
    })

similarity_summary = (
    pd.DataFrame(results)
    .sort_values("Best_similarity", ascending=False)
)

display(similarity_summary)
```

	Allele	Best_similarity	Mean_top10_similarity
3	ALK T1151M_NA	0.157305	0.138056
4	ALK WT_NA	0.150285	0.112098
2	ALK R1275Q_NA	0.106348	0.092016
1	ALK F1174L_NA	0.100580	0.094405
0	ALK A1234T_NA	0.073358	0.052884

All ALK alleles were screened against the JUMP perturbation reference set. T1151M exhibited the highest morphology-based similarity to JUMP perturbations (best cosine similarity = 0.157), followed by WT (0.150), whereas A1234T showed the weakest matches (0.073). Based on these results, T1151M was selected for downstream perturbation annotation.

In [19]:
```
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# Extract top JUMP morphology matches for each ALK allele
#
# Rationale:
# After comparing all ALK alleles against the JUMP perturbation space,
# we inspect the top-matching wells for each allele. This helps determ
# whether different ALK alleles retrieve distinct perturbation profile
# or whether the same JUMP wells dominate across alleles.

top_hits_list = []

for allele in alk_scaled.index:
    sims = cosine_similarity(
        alk_scaled.loc[[allele]],
        jump_scaled
    )[0]

    tmp = jump[
```

```
        ["Metadata_Source", "Metadata_Plate", "Metadata_Well"]
    ].copy()

    tmp["Allele"] = allele
    tmp["Similarity"] = sims

    tmp = tmp.sort_values(
        "Similarity",
        ascending=False
    ).head(10)

    top_hits_list.append(tmp)

top_hits_all = pd.concat(
    top_hits_list,
    ignore_index=True
)

display(top_hits_all)
```

	Metadata_Source	Metadata_Plate	Metadata_Well	Allele	Similarity
0	source_7	CP1-SC1-01	M16	ALK A1234T_NA	0.073358
1	source_7	CP1-SC1-01	O21	ALK A1234T_NA	0.067000
2	source_7	CP1-SC1-01	I08	ALK A1234T_NA	0.059600
3	source_7	CP1-SC1-01	J05	ALK A1234T_NA	0.056436
4	source_7	CP1-SC1-01	J07	ALK A1234T_NA	0.054078
5	source_7	CP1-SC1-01	H01	ALK A1234T_NA	0.046565
6	source_7	CP1-SC1-01	E05	ALK A1234T_NA	0.045864
7	source_7	CP1-SC1-01	F15	ALK A1234T_NA	0.045585
8	source_7	CP1-SC1-01	N11	ALK A1234T_NA	0.040848
9	source_7	CP1-SC1-01	O13	ALK A1234T_NA	0.039510
10	source_7	CP1-SC1-01	I08	ALK F1174L_NA	0.100580
11	source_7	CP1-SC1-01	F15	ALK F1174L_NA	0.097521

12	source_7	CP1-SC1-01	M16	ALK F1174L_NA	0.096249
13	source_7	CP1-SC1-01	J11	ALK F1174L_NA	0.095246
14	source_7	CP1-SC1-01	J21	ALK F1174L_NA	0.094262
15	source_7	CP1-SC1-01	H06	ALK F1174L_NA	0.093245
16	source_7	CP1-SC1-01	A06	ALK F1174L_NA	0.092729
17	source_7	CP1-SC1-01	D24	ALK F1174L_NA	0.092487
18	source_7	CP1-SC1-01	P01	ALK F1174L_NA	0.091842
19	source_7	CP1-SC1-01	J05	ALK F1174L_NA	0.089887
20	source_7	CP1-SC1-01	O21	ALK R1275Q_NA	0.106348
21	source_7	CP1-SC1-01	M16	ALK R1275Q_NA	0.105601
22	source_7	CP1-SC1-01	I08	ALK R1275Q_NA	0.092948
23	source_7	CP1-SC1-01	J05	ALK R1275Q_NA	0.091887
24	source_7	CP1-SC1-01	F15	ALK R1275Q_NA	0.090358
25	source_7	CP1-SC1-01	H06	ALK R1275Q_NA	0.089426
26	source_7	CP1-SC1-01	D24	ALK R1275Q_NA	0.088957
27	source_7	CP1-SC1-01	E05	ALK R1275Q_NA	0.086308
28	source_7	CP1-SC1-01	A06	ALK R1275Q_NA	0.085184
29	source_7	CP1-SC1-01	J21	ALK R1275Q_NA	0.083144
30	source_7	CP1-SC1-01	O21	ALK T1151M_NA	0.157305
31	source_7	CP1-SC1-01	J07	ALK T1151M_NA	0.152601

32	source_7	CP1-SC1-01	I08	ALK T1151M_NA	0.142157
33	source_7	CP1-SC1-01	H01	ALK T1151M_NA	0.140843
34	source_7	CP1-SC1-01	N11	ALK T1151M_NA	0.138946
35	source_7	CP1-SC1-01	J11	ALK T1151M_NA	0.136873
36	source_7	CP1-SC1-01	F15	ALK T1151M_NA	0.135889
37	source_7	CP1-SC1-01	D20	ALK T1151M_NA	0.134223
38	source_7	CP1-SC1-01	L07	ALK T1151M_NA	0.121425
39	source_7	CP1-SC1-01	G18	ALK T1151M_NA	0.120298
40	source_7	CP1-SC1-01	O21	ALK WT_NA	0.150285
41	source_7	CP1-SC1-01	B05	ALK WT_NA	0.138183
42	source_7	CP1-SC1-01	M16	ALK WT_NA	0.127608
43	source_7	CP1-SC1-01	H06	ALK WT_NA	0.105270
44	source_7	CP1-SC1-01	P15	ALK WT_NA	0.103711
45	source_7	CP1-SC1-01	I08	ALK WT_NA	0.101110
46	source_7	CP1-SC1-01	P09	ALK WT_NA	0.101108
47	source_7	CP1-SC1-01	J05	ALK WT_NA	0.100652
48	source_7	CP1-SC1-01	F09	ALK WT_NA	0.096667
49	source_7	CP1-SC1-01	M23	ALK WT_NA	0.096382

Cross-dataset similarity search showed that several JUMP wells were repeatedly retrieved across ALK alleles, suggesting that some matches may reflect broad morphological similarity rather than allele-specific drug-like effects. However, T1151M produced the highest similarity scores overall, consistent with its strong deviation from WT in the VarChAMP analysis. This motivated downstream annotation of the top T1151M-matching perturbation wells.

In [20]:
```
import subprocess

# Install only dependencies required for DILI-to-JUMP matching
subprocess.run(
```

```
    ["pip", "install", "boto3", "rdkit", "-q"],
    capture_output=True
)

import boto3
from botocore import UNSIGNED
from botocore.config import Config
import pandas as pd
import io
import urllib.request
from rdkit.Chem.inchi import InchiToInchiKey

# Load JUMP metadata and match OASIS DILI compounds to JUMP identifier
#
# Rationale:
# To connect ALK variant-associated morphology to drug-induced
# phenotypes, we first construct an annotated set of DILI-positive
# compounds that are present in the JUMP Cell Painting library.
#
# OASIS metadata provides compound names and InChI strings, while
# JUMP metadata provides compound identifiers (Metadata_JCP2022) and
# InChIKeys. Matching through InChIKey allows the same compounds to
# be linked across resources.

BUCKET = "cellpainting-gallery"
s3 = boto3.client(
    "s3",
    config=Config(signature_version=UNSIGNED)
)

METADATA_BASE = (
    "https://raw.githubusercontent.com/"
    "jump-cellpainting/datasets/main/metadata"
)

print("Loading JUMP compound and well metadata...")

with urllib.request.urlopen(f"{METADATA_BASE}/compound.csv.gz") as r:
    jump_compounds = pd.read_csv(r, compression="gzip")

jump_compounds = jump_compounds.dropna(
    subset=["Metadata_InChIKey"]
)

with urllib.request.urlopen(f"{METADATA_BASE}/well.csv.gz") as r:
    jump_well = pd.read_csv(r, compression="gzip")

print(f"JUMP compounds: {jump_compounds.shape[0]:,}")
print(f"JUMP wells:     {jump_well.shape[0]:,}")

# DILI-positive compounds previously identified from OASIS/BioMorph an
```

```
DILI_POSITIVE = [
    "Panobinostat", "Romidepsin", "(E/Z)-Belinostat", "Tacrolimus",
    "Troglitazone", "Diclofenac", "Isoniazid", "Tamoxifen",
    "Valproic acid", "Ketoconazole", "Clarithromycin", "Erythromycin",
    "Rifampicin", "Fluconazole", "Ibuprofen", "Acetaminophen",
    "Tetracycline", "Chlorpromazine", "Nitrofurantoin", "Sulindac",
    "Trovafloxacin", "Nefazodone", "Erdafitinib", "Famoxadone",
    "Indocyanine green"
]

# Retrieve OASIS InChI strings from public S3 biochem metadata

print("\nRetrieving OASIS InChI strings for DILI compounds...")

dili_inchi_map = {}

for batch in ["prod_25", "prod_26", "prod_27", "prod_30"]:
    response = s3.list_objects_v2(
        Bucket=BUCKET,
        Prefix=f"cpg0037-oasis/axiom/workspace/metadata/{batch}/"
    )

    for obj in response.get("Contents", []):
        if "biochem.parquet" not in obj["Key"]:
            continue

        data_obj = s3.get_object(
            Bucket=BUCKET,
            Key=obj["Key"]
        )

        biochem = pd.read_parquet(
            io.BytesIO(data_obj["Body"].read())
        )

        dili_rows = (
            biochem[
                biochem["compound_name"].isin(DILI_POSITIVE)
                & biochem["compound_inchi"].notna()
            ][["compound_name", "compound_inchi"]]
            .drop_duplicates()
        )

        for _, row in dili_rows.iterrows():
            dili_inchi_map.setdefault(
                row["compound_name"],
                row["compound_inchi"]
            )

print(f"DILI compounds with InChI: {len(dili_inchi_map)} / {len(DILI_P

# Convert InChI to InChIKey
```

```
dili_inchikey_map = {}

for name, inchi in dili_inchi_map.items():
    key = InchiToInchiKey(inchi)

    if key:
        dili_inchikey_map[name] = key

print(f"DILI compounds with computed InChIKey: {len(dili_inchikey_map)

# Match DILI compounds to JUMP compound identifiers

matched = []

for name, inchikey in dili_inchikey_map.items():
    exact = jump_compounds[
        jump_compounds["Metadata_InChIKey"] == inchikey
    ]

    if len(exact) > 0:
        for _, row in exact.iterrows():
            matched.append({
                "compound_name": name,
                "Metadata_JCP2022": row["Metadata_JCP2022"],
                "match_type": "exact"
            })

    else:
        first_block = inchikey.split("-")[0]

        partial = jump_compounds[
            jump_compounds["Metadata_InChIKey"].str.startswith(
                first_block,
                na=False
            )
        ]

        if len(partial) > 0:
            for _, row in partial.iterrows():
                matched.append({
                    "compound_name": name,
                    "Metadata_JCP2022": row["Metadata_JCP2022"],
                    "match_type": "partial"
                })

matched_df = pd.DataFrame(matched)

print(
    f"\nDILI compounds matched in JUMP: "
    f"{matched_df['compound_name'].nunique()} / {len(DILI_POSITIVE)}"
)
```

```
print(
    "Exact matches:",
    (matched_df["match_type"] == "exact").sum()
)

print(
    "Partial matches:",
    (matched_df["match_type"] == "partial").sum()
)

display(matched_df.head())
```

```
Loading JUMP compound and well metadata...
JUMP compounds: 115,795
JUMP wells:     1,151,808

Retrieving OASIS InChI strings for DILI compounds...
DILI compounds with InChI: 25 / 25
DILI compounds with computed InChIKey: 25

DILI compounds matched in JUMP: 16 / 25
Exact matches: 8
Partial matches: 8
```

	compound_name	Metadata_JCP2022	match_type
0	Fluconazole	JCP2022_078005	exact
1	Trovafloxacin	JCP2022_101487	partial
2	Nefazodone	JCP2022_095731	exact
3	Ibuprofen	JCP2022_029646	exact
4	Erythromycin	JCP2022_089954	partial

To investigate whether ALK variant-associated morphologies resemble drug-induced toxicity phenotypes, DILI-positive compounds previously identified in the OASIS analysis were mapped to the JUMP Cell Painting reference dataset. Of 25 hepatotoxic compounds, 16 were successfully matched to JUMP perturbations (8 exact chemical matches and 8 connectivity-level matches), providing a set of annotated toxicity-associated morphology profiles for downstream comparison.

In [21]:
```
from pathlib import Path
import os
import pandas as pd
import urllib.request

BASE_DIR = Path("/Users/dr.parul/Downloads/parul_carpenter_project")
JUMP_COMPOUND_DIR = BASE_DIR / "data" / "jump_profiles"
JUMP_CRISPR_DIR = BASE_DIR / "data" / "jump_crispr"
```

```
METADATA_BASE = "https://raw.githubusercontent.com/jump-cellpainting/d

# Load plate and CRISPR metadata required for DILI and gene-level anno
#
# Rationale:
# Plate metadata is needed to identify which JUMP plates contain DILI
# compounds. CRISPR metadata is needed to map JUMP perturbation IDs to
# gene symbols for gene-level morphology matching.

with urllib.request.urlopen(f"{METADATA_BASE}/plate.csv.gz") as r:
    jump_plate = pd.read_csv(r, compression="gzip")

with urllib.request.urlopen(f"{METADATA_BASE}/crispr.csv.gz") as r:
    jump_crispr = pd.read_csv(r, compression="gzip")

print("JUMP plate metadata:", jump_plate.shape)
print("JUMP CRISPR metadata:", jump_crispr.shape)
print(jump_crispr.columns.tolist())
```

```
JUMP plate metadata: (2525, 4)
JUMP CRISPR metadata: (7977, 3)
['Metadata_JCP2022', 'Metadata_NCBI_Gene_ID', 'Metadata_Symbol']
```

In [22]:

```
# Build annotated DILI-JUMP compound profile set
#
# Rationale:
# We identify JUMP wells containing DILI-positive compounds matched
# from OASIS, select source_7 plates enriched for these compounds,
# load their Cell Painting profiles, and annotate wells with compound
# names. This creates the compound/toxicity reference set for matching
# ALK variant morphologies.

dili_wells_jump = (
    jump_well[
        jump_well["Metadata_JCP2022"].isin(matched_df["Metadata_JCP202
    ]
    .merge(
        jump_plate[["Metadata_Source", "Metadata_Plate", "Metadata_Bat
        on=["Metadata_Source", "Metadata_Plate"],
        how="left"
    )
    .merge(
        matched_df[["compound_name", "Metadata_JCP2022", "match_type"]
        on="Metadata_JCP2022",
        how="left"
    )
)

source7_plates = (
    dili_wells_jump[
        dili_wells_jump["Metadata_Source"] == "source_7"
    ]
```

```
    .groupby("Metadata_Plate")
    .agg(
        n_compounds=("compound_name", "nunique"),
        compounds=("compound_name", lambda x: sorted(set(x.dropna())))
    )
    .reset_index()
    .sort_values("n_compounds", ascending=False)
)

top5_plates = source7_plates.head(5)["Metadata_Plate"].tolist()

print("Selected source_7 plates enriched for DILI compounds:")
display(source7_plates.head(5))

COMPOUND_BATCH = "20210719_Run1"
compound_plates = {}

for plate in top5_plates:
    local_path = JUMP_COMPOUND_DIR / f"{plate}.parquet"

    if local_path.exists():
        plate_df = pd.read_parquet(local_path)
        print(f"✓ {plate}: loaded from cache {plate_df.shape}")
    else:
        key = (
            f"cpg0016-jump/source_7/workspace/profiles/"
            f"{COMPOUND_BATCH}/{plate}/{plate}.parquet"
        )
        s3.download_file(BUCKET, key, str(local_path))
        plate_df = pd.read_parquet(local_path)
        print(f"✓ {plate}: downloaded {plate_df.shape}")

    compound_plates[plate] = plate_df

jump_compounds_all = pd.concat(compound_plates.values(), ignore_index=

jump_annotated = (
    jump_compounds_all
    .merge(
        jump_well,
        on=["Metadata_Source", "Metadata_Plate", "Metadata_Well"],
        how="left"
    )
    .merge(
        matched_df[["compound_name", "Metadata_JCP2022", "match_type"]
        on="Metadata_JCP2022",
        how="left"
    )
)

dili_jump_profiles = jump_annotated[
    jump_annotated["compound_name"].notna()
```

```
].copy()

print("\nDILI compound wells:", len(dili_jump_profiles))
print("Unique DILI compounds:", dili_jump_profiles["compound_name"].nu

display(
    dili_jump_profiles
    .groupby("compound_name")
    .size()
    .sort_values(ascending=False)
)
```

Selected source_7 plates enriched for DILI compounds:

	Metadata_Plate	n_compounds	compounds
62	CP6-SC1-02	7	[Diclofenac, Erythromycin, Ibuprofen, Isoniazi...
26	CP3-SC1-02	7	[Diclofenac, Erythromycin, Ibuprofen, Isoniazi...
13	CP2-SC1-02	7	[Diclofenac, Erythromycin, Ibuprofen, Isoniazi...
69	CP8-SC1-02	6	[Erythromycin, Ibuprofen, Isoniazid, Sulindac,...
67	CP7-SC1-02	6	[Erythromycin, Ibuprofen, Isoniazid, Sulindac,...

```
✓ CP6-SC1-02: loaded from cache (384, 4765)
✓ CP3-SC1-02: loaded from cache (384, 4765)
✓ CP2-SC1-02: loaded from cache (384, 4765)
✓ CP8-SC1-02: loaded from cache (384, 4765)
✓ CP7-SC1-02: loaded from cache (384, 4765)

DILI compound wells: 33
Unique DILI compounds: 7
compound_name
Erythromycin     5
Ibuprofen        5
Isoniazid        5
Sulindac         5
Tamoxifen        5
Valproic acid    5
Diclofenac       3
dtype: int64
```

In [23]:

```
from sklearn.preprocessing import StandardScaler
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Match ALK variant profiles to DILI compound profiles
#
```

```
# Rationale:
# This tests whether ALK variant-associated morphologies resemble
# hepatotoxic compound-induced phenotypes. All ALK alleles are compare
# against the annotated DILI-JUMP profile set to avoid selecting a
# single variant a priori.

dili_features = sorted(
    set(shared_features).intersection(
        [c for c in dili_jump_profiles.columns if not c.startswith("Me
    )
)

print("Shared VarChAMP-DILI/JUMP features:", len(dili_features))

alk_dili = alk_centroids[dili_features]
dili_matrix = dili_jump_profiles[dili_features]

combined = pd.concat([alk_dili, dili_matrix], axis=0)

scaled = pd.DataFrame(
    StandardScaler().fit_transform(combined),
    index=combined.index,
    columns=dili_features
)

alk_dili_scaled = scaled.iloc[:len(alk_dili)]
dili_scaled = scaled.iloc[len(alk_dili):]

dili_results = []

for allele in alk_dili_scaled.index:
    sims = cosine_similarity(
        alk_dili_scaled.loc[[allele]],
        dili_scaled
    )[0]

    tmp = dili_jump_profiles[
        ["compound_name", "Metadata_Plate", "Metadata_Well", "Metadata
    ].copy()

    tmp["Allele"] = allele
    tmp["Similarity"] = sims

    dili_results.append(tmp)

dili_similarity = pd.concat(dili_results, ignore_index=True)

compound_summary = (
    dili_similarity
    .groupby(["Allele", "compound_name"])
    .agg(
        max_similarity=("Similarity", "max"),
```

```
        mean_similarity=("Similarity", "mean"),
        n_wells=("Similarity", "size")
    )
    .reset_index()
    .sort_values(["Allele", "max_similarity"], ascending=[True, False])
)

display(compound_summary)
```

```
Shared VarChAMP-DILI/JUMP features: 239
```

	Allele	compound_name	max_similarity	mean_similarity	n_wells
5	ALK A1234T_NA	Tamoxifen	-0.391842	-0.500519	5
6	ALK A1234T_NA	Valproic acid	-0.410338	-0.498248	5
4	ALK A1234T_NA	Sulindac	-0.439117	-0.515450	5
2	ALK A1234T_NA	Ibuprofen	-0.450902	-0.500702	5
3	ALK A1234T_NA	Isoniazid	-0.466956	-0.520438	5
1	ALK A1234T_NA	Erythromycin	-0.471348	-0.525940	5
0	ALK A1234T_NA	Diclofenac	-0.538090	-0.568262	3
12	ALK F1174L_NA	Tamoxifen	-0.444078	-0.490848	5
13	ALK F1174L_NA	Valproic acid	-0.459822	-0.509227	5
10	ALK F1174L_NA	Isoniazid	-0.460668	-0.508719	5
11	ALK F1174L_NA	Sulindac	-0.490796	-0.516733	5
8	ALK F1174L_NA	Erythromycin	-0.491527	-0.516037	5
9	ALK F1174L_NA	Ibuprofen	-0.493204	-0.510239	5
7	ALK F1174L_NA	Diclofenac	-0.520520	-0.567567	3
20	ALK R1275Q_NA	Valproic acid	-0.452505	-0.560141	5
17	ALK	Isoniazid	-0.482389	-0.573699	5

	R1275Q_NA				
16	ALK R1275Q_NA	Ibuprofen	-0.502354	-0.559449	5
19	ALK R1275Q_NA	Tamoxifen	-0.503715	-0.555057	5
18	ALK R1275Q_NA	Sulindac	-0.509497	-0.571463	5
15	ALK R1275Q_NA	Erythromycin	-0.532328	-0.590391	5
14	ALK R1275Q_NA	Diclofenac	-0.613591	-0.646730	3
24	ALK T1151M_NA	Isoniazid	-0.424399	-0.474846	5
27	ALK T1151M_NA	Valproic acid	-0.430010	-0.475361	5
26	ALK T1151M_NA	Tamoxifen	-0.435257	-0.477625	5
21	ALK T1151M_NA	Diclofenac	-0.474222	-0.528990	3
23	ALK T1151M_NA	Ibuprofen	-0.474910	-0.489537	5
25	ALK T1151M_NA	Sulindac	-0.478844	-0.502146	5
22	ALK T1151M_NA	Erythromycin	-0.479897	-0.498738	5
30	ALK WT_NA	Ibuprofen	-0.272794	-0.394544	5
33	ALK WT_NA	Tamoxifen	-0.281720	-0.404710	5
31	ALK WT_NA	Isoniazid	-0.289364	-0.394842	5
32	ALK WT_NA	Sulindac	-0.292129	-0.403085	5
34	ALK WT_NA	Valproic acid	-0.294066	-0.396481	5
28	ALK WT_NA	Diclofenac	-0.303247	-0.404076	3
29	ALK WT_NA	Erythromycin	-0.311239	-0.400162	5

In [24]:
```
# Load and annotate JUMP CRISPR profiles
#
# Rationale:
# Compound matching can suggest drug-induced phenotypic similarity,
# but CRISPR profiles provide gene-level perturbation evidence.
# Comparing ALK variant morphologies to CRISPR knockouts allows us to
```

```
# ask whether variant phenotypes converge on interpretable gene-level
# perturbation signatures.

crispr_files = sorted(JUMP_CRISPR_DIR.glob("*.parquet"))

print("Cached CRISPR profile files:", len(crispr_files))

# Use only shared features available in CRISPR profiles
first_crispr = pd.read_parquet(crispr_files[0])
crispr_features = sorted(
    set(shared_features).intersection(
        [c for c in first_crispr.columns if not c.startswith("Metadata
    )
)

print("Shared VarChAMP-CRISPR features:", len(crispr_features))

metadata_cols = ["Metadata_Source", "Metadata_Plate", "Metadata_Well"]
cols_to_read = metadata_cols + crispr_features

crispr_parts = []

for f in crispr_files:
    tmp = pd.read_parquet(f, columns=cols_to_read)
    crispr_parts.append(tmp)

crispr_all = pd.concat(crispr_parts, ignore_index=True)

crispr_annotated = (
    crispr_all
    .merge(
        jump_well,
        on=["Metadata_Source", "Metadata_Plate", "Metadata_Well"],
        how="left"
    )
    .merge(
        jump_crispr[["Metadata_JCP2022", "Metadata_Symbol"]],
        on="Metadata_JCP2022",
        how="left"
    )
)

crispr_profiles = crispr_annotated[
    crispr_annotated["Metadata_Symbol"].notna()
].copy()

print("Annotated CRISPR profiles:", crispr_profiles.shape)
print("Unique CRISPR genes:", crispr_profiles["Metadata_Symbol"].nuniq

display(crispr_profiles[["Metadata_Plate", "Metadata_Well", "Metadata_
```

```
Cached CRISPR profile files: 148
Shared VarChAMP-CRISPR features: 239
Annotated CRISPR profiles: (51185, 244)
Unique CRISPR genes: 7977
```

	Metadata_Plate	Metadata_Well	Metadata_Symbol
0	CP-CC9-R1-01	A02	non-targeting
1	CP-CC9-R1-01	A03	ARHGEF7
2	CP-CC9-R1-01	A04	ST13
3	CP-CC9-R1-01	A05	GPHN
4	CP-CC9-R1-01	A06	EXT2

In [25]:
```
# Match ALK variant profiles to CRISPR gene perturbation profiles
#
# Rationale:
# This analysis tests whether ALK variant morphologies resemble
# gene-level perturbation signatures in JUMP. Results can reveal
# candidate genes or pathways whose perturbation produces morphology
# similar to specific ALK variants.

alk_crispr = alk_centroids[crispr_features]
crispr_matrix = crispr_profiles[crispr_features]

combined = pd.concat([alk_crispr, crispr_matrix], axis=0)

scaled = pd.DataFrame(
    StandardScaler().fit_transform(combined),
    index=combined.index,
    columns=crispr_features
)

alk_crispr_scaled = scaled.iloc[:len(alk_crispr)]
crispr_scaled = scaled.iloc[len(alk_crispr):]

crispr_results = []

for allele in alk_crispr_scaled.index:
    sims = cosine_similarity(
        alk_crispr_scaled.loc[[allele]],
        crispr_scaled
    )[0]

    tmp = crispr_profiles[
        ["Metadata_Plate", "Metadata_Well", "Metadata_JCP2022", "Metad
    ].copy()

    tmp["Allele"] = allele
    tmp["Similarity"] = sims
```

```
    crispr_results.append(tmp)

crispr_similarity = pd.concat(crispr_results, ignore_index=True)

gene_summary = (
    crispr_similarity
    .groupby(["Allele", "Metadata_Symbol"])
    .agg(
        max_similarity=("Similarity", "max"),
        mean_similarity=("Similarity", "mean"),
        n_wells=("Similarity", "size")
    )
    .reset_index()
    .sort_values(["Allele", "max_similarity"], ascending=[True, False])
)

display(gene_summary.head(50))
```

	Allele	Metadata_Symbol	max_similarity	mean_similarity	n_wells
6989	ALK A1234T_NA	TCEB3B	0.323164	0.081208	5
4332	ALK A1234T_NA	MUC1	0.278437	0.091494	5
6925	ALK A1234T_NA	TAF13	0.274921	0.110189	5
3186	ALK A1234T_NA	HOXB8	0.271381	0.073179	5
1467	ALK A1234T_NA	CPB1	0.270178	0.118240	5
2336	ALK A1234T_NA	FBXW12	0.266787	0.088527	5
7601	ALK A1234T_NA	USP45	0.261998	0.107125	5
7285	ALK A1234T_NA	TRIM47	0.258594	0.108013	5
6932	ALK A1234T_NA	TAF2	0.256394	0.086255	10
7476	ALK A1234T_NA	UBE3A	0.255488	0.115239	5
7506	ALK A1234T_NA	UFC1	0.255169	0.019291	10
813	ALK A1234T_NA	BARD1	0.254109	0.071509	10

1307	ALK A1234T_NA	CHRM4	0.253182	0.112429	5
5101	ALK A1234T_NA	PHF21A	0.253077	0.023181	5
5160	ALK A1234T_NA	PIK3R4	0.250605	0.170659	5
1184	ALK A1234T_NA	CDCA3	0.250342	0.120139	5
2703	ALK A1234T_NA	GJD2	0.250196	0.098103	5
5262	ALK A1234T_NA	PLK2	0.249582	0.090694	5
5915	ALK A1234T_NA	RFPL2	0.248784	0.085990	5
969	ALK A1234T_NA	CA10	0.248349	0.117729	7
6699	ALK A1234T_NA	SOX17	0.247231	-0.009781	7
752	ALK A1234T_NA	ATR	0.245403	0.161693	5
7111	ALK A1234T_NA	TLR5	0.244597	0.123214	7
5376	ALK A1234T_NA	POU3F1	0.244449	0.081633	5
5926	ALK A1234T_NA	RFX7	0.244413	0.111951	5
4943	ALK A1234T_NA	PBXIP1	0.244369	0.028880	7
5757	ALK A1234T_NA	RAB2B	0.243218	0.099710	7
6940	ALK A1234T_NA	TAF7	0.242664	0.120288	5
1392	ALK A1234T_NA	CLOCK	0.242636	0.031474	7
5670	ALK A1234T_NA	PTPMT1	0.242287	0.048841	5
4188	ALK A1234T_NA	MKRN3	0.241437	0.072857	5
2255	ALK A1234T_NA	FAHD2B	0.240717	0.144223	7

3112	ALK A1234T_NA	HIVEP3	0.239720	0.063086	5
7456	ALK A1234T_NA	UBE2I	0.239649	0.116513	5
5378	ALK A1234T_NA	POU3F3	0.239509	0.091572	5
5568	ALK A1234T_NA	PRPS1L1	0.239464	0.064913	5
7606	ALK A1234T_NA	USP5	0.239261	0.027446	5
630	ALK A1234T_NA	ASB9	0.237732	0.088341	10
3602	ALK A1234T_NA	KCNK1	0.235620	0.057107	5
1225	ALK A1234T_NA	CDX2	0.235601	-0.003372	5
2084	ALK A1234T_NA	ELF1	0.235445	0.092562	5
3580	ALK A1234T_NA	KCNH7	0.235431	0.039457	5
7577	ALK A1234T_NA	USP21	0.234372	0.092927	5
5066	ALK A1234T_NA	PGC	0.233984	0.109352	5
5868	ALK A1234T_NA	RBP3	0.233010	0.002944	7
617	ALK A1234T_NA	ASB13	0.232637	0.045287	5
4163	ALK A1234T_NA	MGLL	0.231748	-0.000564	12
133	ALK A1234T_NA	ACR	0.231148	0.111701	5
7297	ALK A1234T_NA	TRIM6	0.230740	0.097176	5
2605	ALK A1234T_NA	GANC	0.229936	0.149473	7

In [26]:
```
# Compare top CRISPR similarity across ALK alleles
#
# Rationale:
```

```
# Identify which ALK variant produces the strongest gene-level
# morphology matches in the JUMP CRISPR reference dataset.

crispr_summary = (
    gene_summary
    .groupby("Allele")
    .agg(
        Best_CRISPR_match=("max_similarity", "max"),
        Mean_top10_CRISPR=("max_similarity",
                           lambda x: x.nlargest(10).mean())
    )
    .sort_values("Best_CRISPR_match", ascending=False)
)

display(crispr_summary)
```

Allele	Best_CRISPR_match	Mean_top10_CRISPR
ALK A1234T_NA	0.323164	0.271734
ALK WT_NA	0.268784	0.254878
ALK T1151M_NA	0.250196	0.222318
ALK R1275Q_NA	0.219946	0.200628
ALK F1174L_NA	0.186419	0.174005

In [27]:
```
# Extract top CRISPR matches for each ALK allele
#
# Rationale:
# The previous analysis identified allele-level differences in CRISPR
# similarity. Here we retrieve the highest-scoring gene perturbations
# for each allele to determine whether specific biological processes
# emerge from the morphology matching analysis.

top_genes = (
    gene_summary
    .sort_values(
        ["Allele", "max_similarity"],
        ascending=[True, False]
    )
    .groupby("Allele")
    .head(10)
)

display(top_genes)
```

	Allele	Metadata_Symbol	max_similarity	mean_similarity	n_wells
6989	ALK	TCEB3B	0.323164	0.081208	5

	A1234T_NA				
4332	ALK A1234T_NA	MUC1	0.278437	0.091494	5
6925	ALK A1234T_NA	TAF13	0.274921	0.110189	5
3186	ALK A1234T_NA	HOXB8	0.271381	0.073179	5
1467	ALK A1234T_NA	CPB1	0.270178	0.118240	5
2336	ALK A1234T_NA	FBXW12	0.266787	0.088527	5
7601	ALK A1234T_NA	USP45	0.261998	0.107125	5
7285	ALK A1234T_NA	TRIM47	0.258594	0.108013	5
6932	ALK A1234T_NA	TAF2	0.256394	0.086255	10
7476	ALK A1234T_NA	UBE3A	0.255488	0.115239	5
12078	ALK F1174L_NA	MED17	0.186419	0.127903	5
10163	ALK F1174L_NA	ETF1	0.179549	0.122250	5
15461	ALK F1174L_NA	UBL5	0.177292	0.113086	5
12073	ALK F1174L_NA	MED12	0.176250	0.095841	5
11063	ALK F1174L_NA	HEXB	0.172582	0.060064	5
14580	ALK F1174L_NA	SLCO2B1	0.171034	0.013051	7
14636	ALK F1174L_NA	SNAPC2	0.170706	0.108305	5
13457	ALK F1174L_NA	PQBP1	0.170155	0.048335	5
8401	ALK F1174L_NA	ALPK3	0.169248	0.108206	5
12328	ALK F1174L_NA	MYCBP	0.166816	0.006763	5
	ALK				

19859	R1275Q_NA	LRRC41	0.219946	0.063674	5
17230	ALK R1275Q_NA	CHCHD3	0.217815	0.080339	5
20305	ALK R1275Q_NA	MYCBP	0.209766	0.063533	5
19127	ALK R1275Q_NA	HOXA4	0.202073	0.064432	5
23875	ALK R1275Q_NA	ZNF593	0.198425	0.070873	5
19933	ALK R1275Q_NA	MAP2K7	0.194385	0.139707	5
19485	ALK R1275Q_NA	JUN	0.191918	0.133406	5
21076	ALK R1275Q_NA	PHYHD1	0.191377	0.115858	7
21781	ALK R1275Q_NA	RAP1GAP	0.190535	0.100725	7
20050	ALK R1275Q_NA	MED12	0.190036	0.071188	5
31166	ALK T1151M_NA	TRAIP	0.250196	0.081087	5
29036	ALK T1151M_NA	PHF5A	0.232840	0.121192	10
27017	ALK T1151M_NA	HEXB	0.223666	0.102801	5
26154	ALK T1151M_NA	F10	0.223509	0.059992	5
31218	ALK T1151M_NA	TRIM49B	0.221039	0.114485	5
26621	ALK T1151M_NA	GJA4	0.220502	0.068781	12
28341	ALK T1151M_NA	NAALADL1	0.220231	0.063882	7
30266	ALK T1151M_NA	SIN3A	0.213116	0.138329	5
29088	ALK T1151M_NA	PIK3R1	0.211870	0.074965	5
29346	ALK T1151M_NA	PPIL2	0.206207	0.054205	5

32472	ALK WT_NA	ARHGEF10	0.268784	0.140163	5
36075	ALK WT_NA	MGST2	0.264805	0.070850	6
31911	ALK WT_NA	A4GNT	0.258790	0.077272	6
31981	ALK WT_NA	ABHD2	0.256825	0.029919	12
32560	ALK WT_NA	ASPHD2	0.255450	0.075529	5
33599	ALK WT_NA	DCLRE1A	0.254228	0.085470	6
39459	ALK WT_NA	UQCR11	0.249756	0.065545	6
33899	ALK WT_NA	ECI2	0.248020	0.130926	5
38072	ALK WT_NA	SAE1	0.247846	0.023434	10
39883	ALK WT_NA	no-guide	0.244275	0.039503	4724

In [31]:
```
# Gene Ontology over-representation analysis of top CRISPR morphology
#
# Rationale:
# We use ORA rather than GSEA because our genes are ranked by
# morphology-based CRISPR similarity, not by differential expression.
# ORA asks whether the top CRISPR knockout genes whose profiles
# resemble each ALK variant are enriched for known biological processe

import subprocess
subprocess.run(["pip", "install", "gseapy", "-q"], capture_output=True

import gseapy as gp
import pandas as pd

TOP_N = 200
go_ora_results = []

for allele in gene_summary["Allele"].unique():

    top_genes = (
        gene_summary[
            gene_summary["Allele"] == allele
        ]
        .sort_values("max_similarity", ascending=False)
```

```
        .head(TOP_N)["Metadata_Symbol"]
        .dropna()
        .drop_duplicates()
        .tolist()
    )

    print(f"Running GO ORA for {allele}: {len(top_genes)} genes")

    enr = gp.enrichr(
        gene_list=top_genes,
        gene_sets=["GO_Biological_Process_2023"],
        organism="human",
        outdir=None,
        verbose=False
    )

    res = enr.results.copy()
    res["Allele"] = allele
    res["Top_N_genes"] = TOP_N
    go_ora_results.append(res)

go_ora_all = pd.concat(go_ora_results, ignore_index=True)

go_ora_summary = (
    go_ora_all
    .sort_values(["Allele", "Adjusted P-value"])
)

display(
    go_ora_summary[
        [
            "Allele",
            "Term",
            "Overlap",
            "P-value",
            "Adjusted P-value",
            "Combined Score",
            "Genes"
        ]
    ]
    .groupby("Allele")
    .head(10)
)
```

```
Running GO ORA for ALK A1234T_NA: 200 genes
Running GO ORA for ALK F1174L_NA: 200 genes
Running GO ORA for ALK R1275Q_NA: 200 genes
Running GO ORA for ALK T1151M_NA: 200 genes
Running GO ORA for ALK WT_NA: 200 genes
```

	Allele	Term	Overlap	P-value	Adjusted P-value	Comb S
	ALK	Protein Modification		2.710896e-	4.077188e-	

0	A1234T_NA	Process (GO:0036211)	27/711	09	06	86.039
1	ALK A1234T_NA	Protein Ubiquitination (GO:0016567)	20/434	1.550319e-08	1.165840e-05	93.559
2	ALK A1234T_NA	Proteolysis (GO:0006508)	17/330	3.950773e-08	1.980654e-05	98.59
3	ALK A1234T_NA	Protein Polyubiquitination (GO:0000209)	14/226	6.835268e-08	2.570061e-05	114.74
4	ALK A1234T_NA	Protein Modification By Small Protein Conjugat...	17/364	1.618161e-07	4.867430e-05	81.433
5	ALK A1234T_NA	Phosphorylation (GO:0016310)	18/429	3.376749e-07	8.464385e-05	69.524
6	ALK A1234T_NA	Protein Phosphorylation (GO:0006468)	19/500	7.079766e-07	1.521138e-04	59.704
7	ALK A1234T_NA	Regulation Of Transcription By RNA Polymerase ...	42/2028	3.705957e-06	6.967199e-04	29.817
8	ALK A1234T_NA	Regulation Of DNA-templated Transcription (GO:...	40/1922	5.926865e-06	9.904450e-04	28.647
9	ALK A1234T_NA	Positive Regulation Of DNA-templated Transcrip...	29/1243	1.782751e-05	2.681257e-03	28.390
1504	ALK F1174L_NA	Protein Ubiquitination (GO:0016567)	31/434	7.284514e-18	8.763270e-15	348.393
1505	ALK F1174L_NA	Regulation Of DNA-templated Transcription (GO:...	61/1922	8.988121e-17	5.406355e-14	156.299
1506	ALK F1174L_NA	Positive Regulation Of DNA-templated Transcrip...	14/56	2.535890e-16	1.016892e-13	1271.55
1507	ALK F1174L_NA	Positive Regulation Of Transcription Initiatio...	13/52	3.043903e-15	9.154538e-13	1177.404
1508	ALK F1174L_NA	Regulation Of Transcription Initiation By RNA ...	13/57	1.123521e-14	2.703191e-12	1002.58
	ALK	Transcription Initiation At RNA		2.521916e-	5.056442e-	

1509	F1174L_NA	Polymerase II ...	14/76	14	12	750.284
1510	ALK F1174L_NA	Regulation Of Transcription By RNA Polymerase ...	58/2028	6.577819e-14	1.130445e-11	112.20
1511	ALK F1174L_NA	RNA Polymerase II Preinitiation Complex Assemb...	12/56	2.725448e-13	4.098393e-11	829.149
1512	ALK F1174L_NA	DNA-templated Transcription Initiation (GO:000...	12/59	5.323227e-13	6.403841e-11	758.148
1513	ALK F1174L_NA	Transcription Preinitiation Complex Assembly (...	12/59	5.323227e-13	6.403841e-11	758.148
2707	ALK R1275Q_NA	Positive Regulation Of DNA-templated Transcrip...	7/56	1.378939e-06	2.320754e-03	197.278
2708	ALK R1275Q_NA	Regulation Of Transcription By RNA Polymerase ...	41/2028	8.954747e-06	5.418278e-03	26.869
2709	ALK R1275Q_NA	Regulation Of Cellular Response To Stress (GO:...	8/107	1.215571e-05	5.418278e-03	93.842
2710	ALK R1275Q_NA	Positive Regulation Of Transcription Initiatio...	6/52	1.287767e-05	5.418278e-03	149.549
2711	ALK R1275Q_NA	RNA Polymerase II Preinitiation Complex Assemb...	6/56	1.986744e-05	5.636876e-03	132.261
2712	ALK R1275Q_NA	Regulation Of Transcription Initiation By RNA ...	6/57	2.202142e-05	5.636876e-03	128.428
2713	ALK R1275Q_NA	Transcription Preinitiation Complex Assembly (...	6/59	2.689336e-05	5.636876e-03	121.266
2714	ALK R1275Q_NA	Positive Regulation Of Signaling Receptor Acti...	5/36	2.786810e-05	5.636876e-03	171.495
2715	ALK R1275Q_NA	Cellular Response To Chemical Stress (GO:0062197)	7/89	3.102233e-05	5.636876e-03	90.535
	ALK	Regulation Of DNA-		3.349302e-	5.636876e-	

2716	R1275Q_NA	templated Transcription (GO:...	38/1922	05	03	22.984
4390	ALK T1151M_NA	Positive Regulation Of DNA-templated Transcrip...	49/1243	4.949620e-17	4.676819e-14	189.852
4391	ALK T1151M_NA	Regulation Of Transcription By RNA Polymerase ...	63/2028	6.598009e-17	4.676819e-14	155.503
4392	ALK T1151M_NA	Regulation Of DNA-templated Transcription (GO:...	61/1922	8.988121e-17	4.676819e-14	156.299
4393	ALK T1151M_NA	Positive Regulation Of Transcription By RNA Po...	38/938	1.300060e-13	5.073484e-11	146.15
4394	ALK T1151M_NA	Positive Regulation Of Nucleic Acid-Templated ...	22/557	4.524880e-08	1.412668e-05	75.264
4395	ALK T1151M_NA	Transcription By RNA Polymerase II (GO:0006366)	13/192	7.221586e-08	1.807357e-05	125.304
4396	ALK T1151M_NA	Protein Ubiquitination (GO:0016567)	19/434	8.104740e-08	1.807357e-05	80.063
4397	ALK T1151M_NA	Protein Modification By Small Protein Conjugat...	17/364	1.618161e-07	3.157438e-05	81.433
4398	ALK T1151M_NA	DNA-templated Transcription (GO:0006351)	12/199	8.067366e-07	1.299644e-04	93.927
4399	ALK T1151M_NA	Regulation Of Gene Expression (GO:0010468)	30/1127	9.017634e-07	1.299644e-04	41.877
5951	ALK WT_NA	Polysaccharide Biosynthetic Process (GO:0000271)	4/12	4.511014e-06	6.464283e-03	621.478
5952	ALK WT_NA	Organic Anion Transport (GO:0015711)	7/89	3.102233e-05	1.598895e-02	90.535
5953	ALK WT_NA	Heparan Sulfate Proteoglycan Biosynthetic Proc...	3/7	3.347303e-05	1.598895e-02	776.623
5954	ALK WT_NA	Heparan Sulfate Proteoglycan	4/23	7.402942e-05	2.652104e-02	202.081

		Biosynthetic Proc...				
5955	ALK WT_NA	Pyrimidine Deoxyribonucleotide Catabolic Proce...	3/12	2.027670e-04	4.753592e-02	284.757
5956	ALK WT_NA	Sodium-Independent Organic Anion Transport (GO...	3/13	2.616581e-04	4.753592e-02	248.584
5957	ALK WT_NA	Heparan Sulfate Proteoglycan Metabolic Process...	3/13	2.616581e-04	4.753592e-02	248.584
5958	ALK WT_NA	Response To Cytokine (GO:0034097)	7/125	2.653785e-04	4.753592e-02	49.814
5959	ALK WT_NA	DNA Modification (GO:0006304)	3/17	6.040394e-04	8.704674e-02	159.519
5960	ALK WT_NA	Ras Protein Signal Transduction (GO:0007265)	7/144	6.236471e-04	8.704674e-02	38.416

In [32]:
```
# Create an interpretable top-5 CRISPR match table
#
# Rationale:
# Gene symbols alone can be difficult to interpret. This table
# summarizes the strongest morphology-matching CRISPR perturbations
# for each ALK allele together with similarity scores.

top5_crispr = (
    gene_summary
    .sort_values(
        ["Allele", "max_similarity"],
        ascending=[True, False]
    )
    .groupby("Allele")
    .head(5)
    .copy()
)

top5_crispr = top5_crispr.merge(
    jump_crispr[
        ["Metadata_Symbol", "Metadata_NCBI_Gene_ID"]
    ].drop_duplicates(),
    on="Metadata_Symbol",
    how="left"
)

display(
    top5_crispr[
```

```
        [
            "Allele",
            "Metadata_Symbol",
            "Metadata_NCBI_Gene_ID",
            "max_similarity",
            "mean_similarity"
        ]
    ]
)
```

	Allele	Metadata_Symbol	Metadata_NCBI_Gene_ID	max_similarity	mea
0	ALK A1234T_NA	TCEB3B	51224.0	0.323164	
1	ALK A1234T_NA	MUC1	4582.0	0.278437	
2	ALK A1234T_NA	TAF13	6884.0	0.274921	
3	ALK A1234T_NA	HOXB8	3218.0	0.271381	
4	ALK A1234T_NA	CPB1	1360.0	0.270178	
5	ALK F1174L_NA	MED17	9440.0	0.186419	
6	ALK F1174L_NA	ETF1	2107.0	0.179549	
7	ALK F1174L_NA	UBL5	59286.0	0.177292	
8	ALK F1174L_NA	MED12	9968.0	0.176250	
9	ALK F1174L_NA	HEXB	3074.0	0.172582	
10	ALK R1275Q_NA	LRRC41	10489.0	0.219946	
11	ALK R1275Q_NA	CHCHD3	54927.0	0.217815	
12	ALK R1275Q_NA	MYCBP	26292.0	0.209766	
13	ALK R1275Q_NA	HOXA4	3201.0	0.202073	
14	ALK R1275Q_NA	ZNF593	51042.0	0.198425	
15	ALK	TRAIP	10293.0	0.250196	

	T1151M_NA			
16	ALK T1151M_NA	PHF5A	84844.0	0.232840
17	ALK T1151M_NA	HEXB	3074.0	0.223666
18	ALK T1151M_NA	F10	2159.0	0.223509
19	ALK T1151M_NA	TRIM49B	283116.0	0.221039
20	ALK WT_NA	ARHGEF10	9639.0	0.268784
21	ALK WT_NA	MGST2	4258.0	0.264805
22	ALK WT_NA	A4GNT	51146.0	0.258790
23	ALK WT_NA	ABHD2	11057.0	0.256825
24	ALK WT_NA	ASPHD2	57168.0	0.255450

In [33]:
```
# Create an interpretable CRISPR phenotype-matching table
#
# Rationale:
# CRISPR morphology matches indicate that an ALK variant profile
# resembles the phenotype produced by knockout of a specific gene.
# To make this biologically interpretable, we summarize the top
# CRISPR matches for each ALK allele and add a short annotation of
# the cellular process represented by each matched gene.

gene_function_map = {
    "TCEB3B": "Transcription elongation / RNA polymerase II regulation
    "MUC1": "Cell-surface mucin; adhesion, signaling, epithelial stres
    "TAF13": "TFIID complex; RNA polymerase II transcription initiatio
    "HOXB8": "Developmental transcription factor",
    "CPB1": "Protease; protein processing",

    "MED17": "Mediator complex; RNA polymerase II transcription regula
    "ETF1": "Translation termination factor",
    "UBL5": "Ubiquitin-like protein; spliceosome/protein quality contr
    "MED12": "Mediator kinase module; transcriptional regulation",
    "HEXB": "Lysosomal enzyme; glycosphingolipid metabolism",

    "LRRC41": "Protein regulation; limited functional annotation",
    "CHCHD3": "Mitochondrial cristae organization",
    "MYCBP": "MYC-binding protein; MYC-associated transcriptional regu
    "HOXA4": "Developmental transcription factor",
```

```
    "ZNF593": "Zinc-finger protein; transcription-associated function"

    "TRAIP": "DNA replication stress response and genome stability",
    "PHF5A": "Spliceosome component; RNA splicing and gene expression
    "F10": "Coagulation factor X; serine protease",
    "TRIM49B": "TRIM-family protein; likely ubiquitin-related function

    "ARHGEF10": "Rho GTPase regulation; cytoskeletal signaling",
    "MGST2": "Glutathione metabolism and oxidative stress response",
    "A4GNT": "Glycosyltransferase; mucin/glycan biosynthesis",
    "ABHD2": "Lipid metabolism / hydrolase activity",
    "ASPHD2": "Aspartate beta-hydroxylase domain protein; limited anno
}

top5_crispr_interpretable = (
    gene_summary
    .sort_values(
        ["Allele", "max_similarity"],
        ascending=[True, False]
    )
    .groupby("Allele")
    .head(5)
    .copy()
)

top5_crispr_interpretable["Matched_CRISPR_KO_phenotype"] = (
    top5_crispr_interpretable["Metadata_Symbol"]
    .map(gene_function_map)
)

top5_crispr_interpretable = top5_crispr_interpretable[
    [
        "Allele",
        "Metadata_Symbol",
        "Matched_CRISPR_KO_phenotype",
        "max_similarity",
        "mean_similarity",
        "n_wells"
    ]
].rename(
    columns={
        "Allele": "ALK_variant",
        "Metadata_Symbol": "Top_CRISPR_KO_gene",
        "max_similarity": "Max_cosine_similarity",
        "mean_similarity": "Mean_cosine_similarity",
        "n_wells": "Number_of_CRISPR_wells"
    }
)

display(top5_crispr_interpretable)
```

ALK_variant	Top_CRISPR_KO_gene	Matched_CRISPR_KO_phenotype	Ma

6989	ALK A1234T_NA	TCEB3B	Transcription elongation / RNA polymerase II r...
4332	ALK A1234T_NA	MUC1	Cell-surface mucin; adhesion, signaling, epith...
6925	ALK A1234T_NA	TAF13	TFIID complex; RNA polymerase II transcription...
3186	ALK A1234T_NA	HOXB8	Developmental transcription factor
1467	ALK A1234T_NA	CPB1	Protease; protein processing
12078	ALK F1174L_NA	MED17	Mediator complex; RNA polymerase II transcript...
10163	ALK F1174L_NA	ETF1	Translation termination factor
15461	ALK F1174L_NA	UBL5	Ubiquitin-like protein; spliceosome/protein qu...
12073	ALK F1174L_NA	MED12	Mediator kinase module; transcriptional regula...
11063	ALK F1174L_NA	HEXB	Lysosomal enzyme; glycosphingolipid metabolism
19859	ALK R1275Q_NA	LRRC41	Protein regulation; limited functional annotation
17230	ALK R1275Q_NA	CHCHD3	Mitochondrial cristae organization
20305	ALK R1275Q_NA	MYCBP	MYC-binding protein; MYC-associated transcript...
19127	ALK R1275Q_NA	HOXA4	Developmental transcription factor
23875	ALK R1275Q_NA	ZNF593	Zinc-finger protein; transcription-associated ...
31166	ALK T1151M_NA	TRAIP	DNA replication stress response and genome sta...
29036	ALK T1151M_NA	PHF5A	Spliceosome component; RNA splicing and gene e...
27017	ALK T1151M_NA	HEXB	Lysosomal enzyme; glycosphingolipid metabolism
26154	ALK T1151M_NA	F10	Coagulation factor X; serine protease
31218	ALK T1151M_NA	TRIM49B	TRIM-family protein; likely ubiquitin-related ...

32472	ALK WT_NA	ARHGEF10	Rho GTPase regulation; cytoskeletal signaling
36075	ALK WT_NA	MGST2	Glutathione metabolism and oxidative stress re...
31911	ALK WT_NA	A4GNT	Glycosyltransferase; mucin/glycan biosynthesis
31981	ALK WT_NA	ABHD2	Lipid metabolism / hydrolase activity
32560	ALK WT_NA	ASPHD2	Aspartate beta-hydroxylase domain protein; lim...

In [34]:
```
import requests
import pandas as pd
import time

ALK_VARIANTS = [
    "F1174L",
    "R1275Q",
    "T1151M",
    "A1234T"
]

records = []

for variant in ALK_VARIANTS:

    query = f"ALK[Gene Name] AND {variant}"

    search_url = (
        "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/"
        "esearch.fcgi"
    )

    search = requests.get(
        search_url,
        params={
            "db": "clinvar",
            "term": query,
            "retmode": "json"
        }
    ).json()

    ids = search["esearchresult"]["idlist"]

    print(f"{variant}: {len(ids)} records")

    records.append({
        "ALK_variant": variant,
        "ClinVar_records_found": len(ids),
```

```
        "ClinVar_IDs": ", ".join(ids)
    })

    time.sleep(1.5)

clinvar_hits = pd.DataFrame(records)

display(clinvar_hits)
```

```
F1174L: 4 records
R1275Q: 1 records
T1151M: 1 records
A1234T: 0 records
```

	ALK_variant	ClinVar_records_found	ClinVar_IDs
0	F1174L	4	823946, 217852, 217851, 217849
1	R1275Q	1	18083
2	T1151M	1	18086
3	A1234T	0	

Final inference:

- **F1174L, R1275Q, and T1151M** all have ClinVar records, so their morphology/CRISPR signals can be compared against known clinical variant annotation.
- **A1234T has no ClinVar record found**, but it showed the strongest CRISPR similarity signal. That makes it interesting as a potentially under-annotated variant where morphology may provide functional evidence beyond existing ClinVar coverage.

ClinVar search recovered records for F1174L, R1275Q, and T1151M, but not A1234T. Notably, A1234T showed the strongest CRISPR morphology-matching signal despite lacking ClinVar annotation, suggesting that image-based profiling may help prioritize variants with limited clinical annotation for further functional investigation.

ClinVar interrogation identified existing clinical records for F1174L, R1275Q, and T1151M, all of which are established ALK oncogenic or resistance-associated variants. In contrast, A1234T lacked a ClinVar record but demonstrated one of the strongest morphological perturbation signatures and the highest CRISPR phenotype similarity scores. This suggests that image-based profiling may provide functional evidence for variants with limited existing clinical annotation.

In [35]:
```
# Evaluate significance of the strongest CRISPR morphology match
```

```
#
# Rationale:
# To determine whether the top CRISPR match for A1234T represents
# an unusually strong phenotype similarity, we compare the maximum
# cosine similarity against the full background distribution of
# similarities across all CRISPR perturbations.
from sklearn.metrics.pairwise import cosine_similarity

query = alk_scaled.loc["ALK A1234T_NA"]

crispr_similarities = cosine_similarity(
    query.values.reshape(1, -1),
    crispr_scaled
)[0]

top_similarity = crispr_similarities.max()

null_mean = crispr_similarities.mean()
null_std  = crispr_similarities.std()

z_score = (top_similarity - null_mean) / null_std

print(f"Null mean: {null_mean:.4f}")
print(f"Null SD:   {null_std:.4f}")
print(f"Top similarity: {top_similarity:.4f}")
print(f"Z-score: {z_score:.2f}")
```

```
Null mean: 0.0159
Null SD:   0.1632
Top similarity: 0.4184
Z-score: 2.47
```

In [36]:

```
# Statistical significance of the best CRISPR match
#
# Rationale:
# Compare the strongest observed CRISPR morphology match
# against the background distribution of similarities across
# all CRISPR perturbations. Larger positive Z-scores indicate
# increasingly unusual phenotype similarity relative to the
# null distribution.

top_similarity = 0.323164   # TCEB3B match for ALK A1234T

null_mean = crispr_similarities.mean()
null_std  = crispr_similarities.std()

z_score = (top_similarity - null_mean) / null_std

print(f"Null mean: {null_mean:.4f}")
print(f"Null SD:   {null_std:.4f}")
print(f"Top match: {top_similarity:.4f}")
print(f"Z-score:   {z_score:.2f}")
```

```
Null mean: 0.0159
Null SD:   0.1632
Top match: 0.3232
Z-score:   1.88
```

A1234T produced reproducible morphological changes and yielded modest CRISPR similarity signals despite lacking ClinVar annotation. However, its strongest CRISPR match (TCEB3B; cosine similarity = 0.323, Z = 1.88) did not represent a statistically strong outlier relative to the background distribution of CRISPR perturbation similarities. Consequently, while A1234T remains an interesting candidate for further investigation, the current evidence is insufficient to establish pathogenicity or confidently assign a specific biological mechanism.

In contrast, the known ALK variants F1174L, R1275Q, and T1151M demonstrated support from existing clinical annotation, coherent pathway enrichment, and biologically interpretable CRISPR perturbation matches. Among these, T1151M emerged as the most distinctive variant, exhibiting the greatest morphological deviation from wild-type ALK together with enrichment of replication stress, transcriptional regulation, and cell-cycle programs.

Definitive resolution of A1234T's functional status will likely require either larger-scale Cell Painting profiling with increased replicate depth or benchmarking against a curated set of variants with established functional classifications. Such analyses are precisely the type of evaluation enabled by the production-scale VarChAMP Batch 7/8 dataset, which was designed to connect variant-induced morphology with known functional impact across a broader spectrum of clinically relevant mutations.

In [38]:
```
import pandas as pd
from pathlib import Path
from scipy.stats import spearmanr

# Compare AlphaMissense predictions with morphology-derived impact sco
#
# Rationale:
# AlphaMissense provides sequence-based pathogenicity predictions,
# while VarChAMP/Cell Painting provides phenotype-based evidence.
# Comparing the two asks whether morphology-derived impact agrees
# with or diverges from sequence-based prediction.

am_tsv = Path(
    "/Users/dr.parul/Downloads/parul_carpenter_project/data/alphamisse
    "AlphaMissense_aa_substitutions.tsv"
)

ALK_UNIPROT = "Q9UM73"
```

```
morphology_scores = pd.DataFrame({
    "ALK_variant": [
        "ALK T1151M_NA",
        "ALK A1234T_NA",
        "ALK F1174L_NA",
        "ALK R1275Q_NA"
    ],
    "Protein_change_short": [
        "T1151M",
        "A1234T",
        "F1174L",
        "R1275Q"
    ],
    "Distance_to_WT": [
        73.253717,
        71.479383,
        67.196268,
        63.605069
    ]
})

morphology_scores["Morphology_rank"] = (
    morphology_scores["Distance_to_WT"]
    .rank(ascending=False, method="min")
    .astype(int)
)

target_variants = morphology_scores["Protein_change_short"].tolist()

alk_am_hits = []

for chunk in pd.read_csv(
    am_tsv,
    sep="\t",
    skiprows=3,
    chunksize=1_000_000
):
    hit = chunk[
        (chunk["uniprot_id"] == ALK_UNIPROT)
        & (chunk["protein_variant"].isin(target_variants))
    ].copy()

    if len(hit) > 0:
        alk_am_hits.append(hit)

alphamissense_alk = pd.concat(
    alk_am_hits,
    ignore_index=True
)

alphamissense_alk = alphamissense_alk.rename(
    columns={
```

```
        "protein_variant": "Protein_change_short",
        "am_pathogenicity": "AlphaMissense_score",
        "am_class": "AlphaMissense_class"
    }
)

comparison = morphology_scores.merge(
    alphamissense_alk[
        [
            "Protein_change_short",
            "AlphaMissense_score",
            "AlphaMissense_class"
        ]
    ],
    on="Protein_change_short",
    how="left"
)

comparison["AlphaMissense_rank"] = (
    comparison["AlphaMissense_score"]
    .rank(ascending=False, method="min")
    .astype(int)
)

display(
    comparison.sort_values(
        "Distance_to_WT",
        ascending=False
    )
)

rho, pval = spearmanr(
    comparison["Distance_to_WT"],
    comparison["AlphaMissense_score"]
)

print(f"Spearman correlation: rho = {rho:.3f}, p = {pval:.3g}")

print("\nMorphology ranking:")
print(
    comparison
    .sort_values("Distance_to_WT", ascending=False)
    [["Protein_change_short", "Distance_to_WT"]]
    .to_string(index=False)
)

print("\nAlphaMissense ranking:")
print(
    comparison
    .sort_values("AlphaMissense_score", ascending=False)
    [["Protein_change_short", "AlphaMissense_score", "AlphaMissense_cl
    .to_string(index=False)
```

```
)
```

	ALK_variant	Protein_change_short	Distance_to_WT	Morphology_rank	AlphaM
0	ALK T1151M_NA	T1151M	73.253717	1	
1	ALK A1234T_NA	A1234T	71.479383	2	
2	ALK F1174L_NA	F1174L	67.196268	3	
3	ALK R1275Q_NA	R1275Q	63.605069	4	

```
Spearman correlation: rho = -0.600, p = 0.4

Morphology ranking:
Protein_change_short  Distance_to_WT
              T1151M       73.253717
              A1234T       71.479383
              F1174L       67.196268
              R1275Q       63.605069

AlphaMissense ranking:
Protein_change_short  AlphaMissense_score AlphaMissense_class
              F1174L               0.9997          pathogenic
              R1275Q               0.9906          pathogenic
              T1151M               0.9373          pathogenic
              A1234T               0.7999          pathogenic
```

AlphaMissense classified all four ALK variants as pathogenic, supporting the conclusion that none of the variants should be interpreted as silent based on sequence context alone. However, AlphaMissense and morphology produced different severity rankings. Morphology ranked T1151M and A1234T as the strongest phenotypic outliers, whereas AlphaMissense ranked F1174L and R1275Q highest. The negative Spearman correlation (ρ = −0.60, p = 0.40; n = 4) should not be overinterpreted because of the small number of variants, but it supports the idea that sequence-based pathogenicity prediction and Cell Painting morphology capture partially distinct dimensions of variant impact.

Discussion

From OASIS DILI Exploration to Variant Interpretation

This project began as an open-ended exploration of the VarChAMP dataset with the goal of identifying genes exhibiting functionally informative morphological variation. An initial attempt to connect OASIS DILI-positive compounds with JUMP

Cell Painting profiles successfully identified overlapping compounds and generated a curated DILI reference set. However, compound-level morphology matching did not reveal strong or biologically interpretable relationships with ALK variant phenotypes. This negative result motivated a shift from compound-centric analysis toward genetic perturbation profiling, ultimately focusing on ALK as a candidate gene exhibiting substantial variant-specific morphological diversity.

ALK Variants Produce Distinct Morphological Phenotypes

All four ALK missense variants examined (T1151M, A1234T, F1174L, and R1275Q) produced measurable morphological perturbations relative to wild-type ALK in the VarChAMP Cell Painting profiles. Quantification of phenotypic impact using Euclidean distance from the wild-type centroid ranked the variants as:

T1151M > A1234T > F1174L > R1275Q

indicating that T1151M and A1234T induced the largest cellular phenotypic deviations.

Importantly, F1174L and R1275Q exhibited strong inter-variant morphological similarity ($r = 0.74$). Given that both variants are established activating ALK mutations associated with neuroblastoma, this observation suggests that Cell Painting captures biologically meaningful relationships between alleles rather than simply measuring distance from wild type. The ability of morphology to recover known functional relationships supports the utility of image-based profiling for variant interpretation.

CRISPR Perturbation Matching Reveals Coherent Biology

To investigate potential mechanisms underlying the observed phenotypes, each ALK variant was compared against the JUMP CRISPR perturbation collection. The highest-scoring CRISPR matches identified genes involved in transcriptional regulation, ubiquitination, proteostasis, and gene-expression control.

Gene Ontology over-representation analysis of the top morphology-matched CRISPR perturbations revealed remarkably coherent biological themes. Across multiple variants, enriched processes included:

- Protein ubiquitination and protein modification
- RNA Polymerase II transcriptional regulation
- Transcription initiation and Mediator complex activity
- Regulation of gene expression
- Cellular stress-response pathways

F1174L and R1275Q were particularly enriched for transcriptional machinery involving Mediator-complex and RNA Polymerase II functions, while T1151M showed additional enrichment for genome maintenance and regulatory pathways. A1234T exhibited enrichment for ubiquitination, phosphorylation, proteolysis, and transcription-related processes, suggesting that its phenotype converges on biologically meaningful cellular programs despite limited prior clinical annotation.

A1234T Emerged as the Most Interesting Unannotated Variant

A1234T proved especially challenging to interpret. Unlike the other ALK variants, it lacked ClinVar annotation and therefore had no established clinical classification. Furthermore, its strongest CRISPR perturbation match (TCEB3B; cosine similarity = 0.323) produced a modest Z-score of 1.88 relative to the background similarity distribution, indicating that the match was suggestive but not exceptionally strong.

Based solely on morphology and CRISPR matching, A1234T could therefore only be considered a candidate variant requiring additional evidence.

The interpretation changed substantially following integration of AlphaMissense predictions. AlphaMissense classified A1234T as pathogenic (score = 0.7999), providing an independent sequence-based line of evidence supporting functional impact. Because A1234T also produced one of the strongest morphological perturbations observed in the dataset, the absence of ClinVar annotation appears more consistent with under-characterisation than with a benign interpretation.

While computational evidence alone cannot establish clinical pathogenicity, the convergence of morphology-based and sequence-based evidence supports prioritisation of A1234T for future functional validation studies.

Morphology and AlphaMissense Capture Different Dimensions of Variant Impact

Comparison of morphology-derived impact scores with AlphaMissense predictions revealed an important insight. Although AlphaMissense classified all four variants as pathogenic, the severity rankings differed substantially.

Morphology ranked the variants:

T1151M > A1234T > F1174L > R1275Q

whereas AlphaMissense ranked them:

F1174L > R1275Q > T1151M > A1234T

The resulting Spearman correlation was negative (ρ = -0.60), although statistical significance could not be assessed meaningfully because only four variants were available for comparison.

This discordance suggests that sequence-based pathogenicity prediction and morphology-based functional profiling capture partially orthogonal aspects of variant biology. AlphaMissense estimates the likelihood that an amino-acid substitution disrupts protein function using evolutionary and structural information. In contrast, Cell Painting measures the downstream cellular consequences of that perturbation.

The observation that T1151M produced the strongest morphological phenotype despite receiving a lower AlphaMissense score than F1174L and R1275Q is therefore consistent with recent findings from pooled image-based variant profiling studies, including VIS-seq (Pendyala et al., 2025), which demonstrated that morphology can provide complementary and, in some contexts, more discriminative information than sequence-based predictors alone.

Limitations

Several limitations should be considered when interpreting these results.

First, the analysis was performed using publicly available centroid-level VarChAMP profiles and a relatively small set of ALK variants, limiting statistical power and preventing robust estimation of uncertainty around effect sizes.

Second, CRISPR matching identifies phenotypic similarity rather than direct mechanistic relationships. Similar morphology does not necessarily imply participation in the same pathway or biological process.

Third, AlphaMissense provides computational pathogenicity predictions rather than clinical classifications. Consequently, agreement between AlphaMissense and morphology should be interpreted as convergent computational evidence rather than proof of pathogenicity.

Finally, morphology-based phenotypes and sequence-based predictions capture different aspects of variant function, making direct comparisons inherently imperfect.

Conclusions and Future Directions

Despite these limitations, the analysis demonstrates the value of morphology-based profiling as a complementary approach for variant interpretation. Starting from an open-ended exploration of VarChAMP, the workflow progressed from compound-level DILI analysis to genetic perturbation matching, pathway interpretation, clinical annotation, and sequence-based prediction.

The results highlight three key findings:

1. ALK variants produce distinct and biologically meaningful morphological phenotypes.
2. Morphology-derived CRISPR matching recovers coherent transcriptional and proteostasis-related biology.
3. A1234T emerges as a high-priority candidate variant because it combines strong morphological impact with AlphaMissense-supported pathogenicity despite lacking ClinVar annotation.

Future work using production-scale VarChAMP datasets, increased replicate depth, single-cell resolution profiling, and direct benchmarking against experimentally validated variants will be necessary to convert these prioritisation signals into confident functional classifications. Nevertheless, the present analysis illustrates how image-based phenotyping can complement sequence-based prediction and help identify potentially important variants that remain under-characterised in existing clinical databases.

The End

In [ ]: